Milestone Two: Data Science Capstone

Module C, ICU Delirium Prediction

Mike Hembree

November 9, 2026

Problem Statement

My capstone is an early warning model for ICU delirium built on MIMIC IV, the de-identified electronic health record database from Beth Israel Deaconess Medical Center (Johnson et al., 2023a). Delirium is common in the ICU, so a model that flags patients early would let staff act sooner. The full database needs credentialing that is still pending, so the MIMIC work here uses the open MIMIC IV Clinical Database Demo, 140 ICU stays from 100 patients, and predicts whether a stay had a positive delirium screen (Johnson et al., 2023b). For a second dataset, I used the ECAMS study from Massachusetts General Hospital, 373 patients with 1189 features pulled from routine EEG recordings and a delirium call from the Confusion Assessment Method (van Sleuwen et al., 2022). The two are close to opposites. MIMIC has few rows and nine simple features, while ECAMS has more patients, but its 1189 features are over three times the number of patients. I go deep on gradient boosting from Week 9 and on K-means with the silhouette score from Week 10, and I cover K nearest neighbors, DBSCAN, and hierarchical clustering more briefly.

Data and Exploratory Analysis

Exploratory work shaped every model choice. The MIMIC table has nine predictors per stay (age, gender, six averaged vitals and the lowest total GCS), four missing values that I fill with the median inside each training fold, and 45 positive stays out of 140. Only 100 patients are behind those 140 stays, so a few patients show up in more than one fold, which probably makes

my validation numbers a little optimistic. The strongest single feature is the lowest GCS, with a correlation of 0.38 with delirium, and the rest are 0.17 or lower. ECAMS has no missing values and 254 delirious patients out of 373 (68 percent), so the positive class is the majority there. Its feature standard deviations run from 0.0012 to over 27,000, and 1058 of the 1189 features have another feature correlated with them above 0.95. Those facts pointed to three things, and all three held up. Scaling would matter a lot for any distance method, a handful of principal components would carry most of the information in ECAMS, and ECAMS would be easier to predict than MIMIC.

Approach, Metrics and Overfitting

My main metric is ROC AUC, which measures how well a model ranks a delirious patient above a non-delirious one at any cutoff, plus recall and precision at the 0.5 cutoff. I score every model two ways. One is repeated stratified five-fold cross-validation with three repeats on all the rows. The other is a stratified 20 percent holdout (28 MIMIC stays with 9 positives, 75 ECAMS patients with 51 positives) with the model fit on the other 80 percent. Imputation, scaling, PCA, and all tuning happen inside the training folds so no fold sees its own validation rows, and tuned models use an inner search inside each outer fold. Since the holdout sets are tiny, I put bootstrap intervals around the test AUC. To watch for overfitting, I compare training and validation AUC for every method. The hyperparameters connect to the data in a direct way. With 112 training stays per fold, k in KNN, tree depth, learning rate, and number of rounds in boosting are all really dials for how much one or two patients are allowed to decide a prediction. For the clustering methods, eps, min_samples, and the linkage decide whether a rare group becomes a cluster or gets thrown out. Outside the course materials, I used the scikit-learn user guide (Pedregosa et al., 2011), Hastie et al. (2009), and the original papers cited in each section.

K Nearest Neighbors (Breadth)

KNN predicts delirium from the k most similar patients, so the metric and the scaling decide almost everything. I compared Euclidean, Manhattan, Chebyshev, and cosine distance for many values of k. Scaling was the biggest effect. The best unscaled model reached a CV AUC of 0.643 on MIMIC and 0.698 on ECAMS, and the scaled ones reached 0.730 and 0.879. The metric mattered on MIMIC, where Manhattan reached 0.767 and Euclidean 0.730, but on ECAMS, cosine, Manhattan, and Euclidean were within 0.005 of each other (0.884, 0.883, and 0.879). Chebyshev was the worst on both. Figure 1 (right) shows the overfitting. At k of 1, training AUC is 1.0 and validation is 0.63 on MIMIC and 0.72 on ECAMS, and the gap shrinks to 0.03 and 0.013 at k of 31. The nested-tuned KNN scored 0.741 on MIMIC and 0.874 on ECAMS, compared with random forests at 0.779 and 0.898. Ten principal components matched all 1189 columns (0.873 against about 0.88), which fits the redundancy in the EDA.

Table 1. Main results on both datasets. The four model rows are ROC AUC from repeated cross-validation (CV) and from the held-out test set.

Method	*MIMIC IV demo*	*ECAMS*
Data	140 stays, 9 features, 32% delirium	373 patients, 1189 features, 68% delirium
KNN, tuned	CV 0.741, test 0.836	CV 0.874, test 0.868
Boosting, tuned	CV 0.759, test 0.918	CV 0.889, test 0.886
Random forest	CV 0.779, test 0.854	CV 0.898, test 0.873
Logistic regression	CV 0.750, test 0.801	CV 0.876, test 0.892
K-means, scaled	k = 9, silhouette 0.171	k = 2, silhouette 0.128
DBSCAN	eps 2.0: 3 clusters, 44% noise	eps 34: 1 cluster, 9% noise
Ward, 4 clusters	delirium rates 27% to 42%	delirium rates 45% to 98%

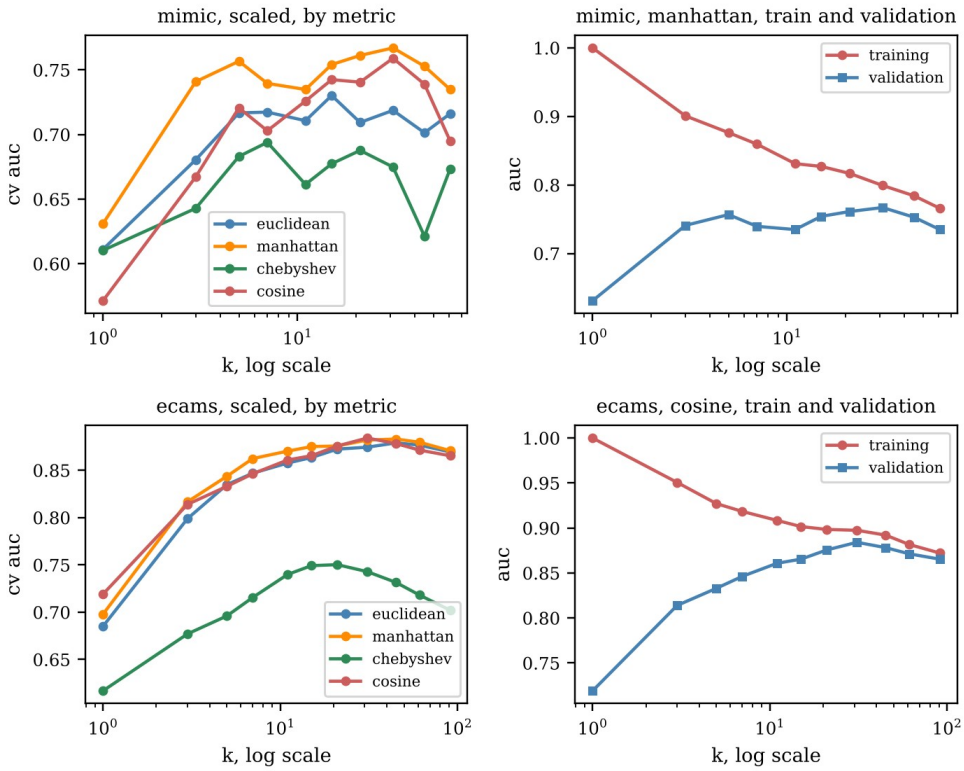


Figure 1. KNN cross-validated AUC by distance metric and k on scaled features (left), and training against validation AUC for the best metric (right), for MIMIC (top) and ECAMS (bottom).

Gradient Boosting (Depth Topic 1)

Gradient boosting builds shallow trees one at a time, each fit to the errors of the ensemble so far (Friedman, 2001). I studied learning rate, rounds, tree depth, subsampling, minimum leaf size, and early stopping. Figure 2 shows validation and training AUC after every round on both datasets. Training AUC ends at or near 1.0 by round 300 in almost every setting (0.98 for the slowest learning rate), so what matters is when validation turns around. On MIMIC, it peaks early and then slides. A learning rate of 0.3 peaks at round 9 (0.786) and has dropped to 0.730 by round 300; a rate of 0.1 peaks at round 30 and ends at 0.740, and only the 0.01 rate is still improving at round 300 (0.777). Every learning rate had about the same best validation AUC, near 0.78, so the learning rate mostly sets how many rounds fit before memorizing starts. Depth 1 trees peaked at 0.781 and depth 5 at 0.759. ECAMS was flatter, and all settings fell between

0.874 and 0.897. There deeper trees were slightly better (depth 5 held 0.893 at 300 rounds where depth 1 fell to 0.877), which is the opposite of MIMIC.

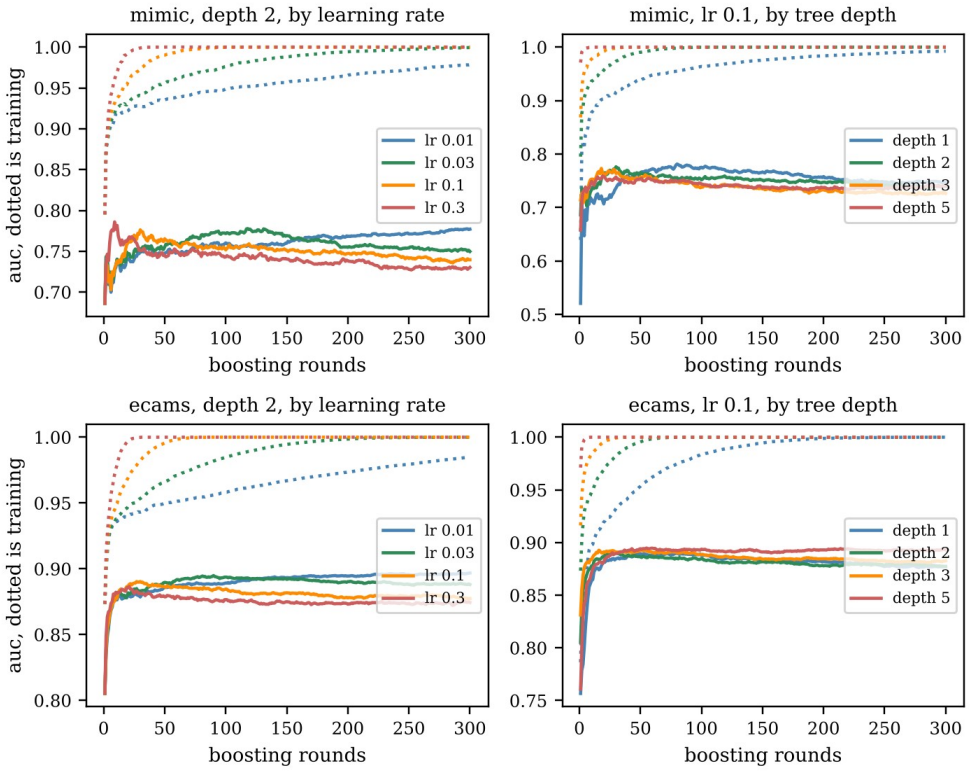


Figure 2. Gradient boosting AUC by round for four learning rates (left) and four tree depths (right), validation as solid lines and training as dotted lines, for MIMIC (top) and ECAMS (bottom).

Regularization options did not remove the gap, since training AUC was 1.0 in every run, but they changed the validation score. On MIMIC, subsampling half of the rows with a minimum leaf of 5 gave 0.770 against 0.735 plain, though the fold standard deviation is near 0.1, so that is not proven. On ECAMS, using every feature at each split (0.867) was worse than sampling the square root of them (0.884), so feature subsampling acts as regularization when the columns are near copies. Early stopping stopped after 25 to 55 trees on MIMIC and 40 to 74 on ECAMS and matched the best fixed setting. Nested tuning gave 0.759 on MIMIC, up from 0.738 for default boosting, and 0.889 on ECAMS, the same as the default 0.888. Boosting never beat the random

forest in cross-validation (0.759 against 0.779 on MIMIC and 0.889 against 0.898 on ECAMS), and on the ECAMS holdout, logistic regression edged it out (0.892 against 0.886), which I did not expect. On the holdout, the tuned MIMIC model scored 0.918 with an interval of 0.81 to 1.00, well above its CV estimate of 0.759, and with only 9 positive stays, I read that as a friendly split.

In Figure 3, permutation importance on MIMIC is concentrated in the lowest GCS, with a drop of 0.20 AUC and then temperature at 0.05, and everything else is under 0.01. On ECAMS, the top feature only drops the AUC by 0.003 because shuffling one of many near copies does little, so that ranking is weak evidence about any single feature.

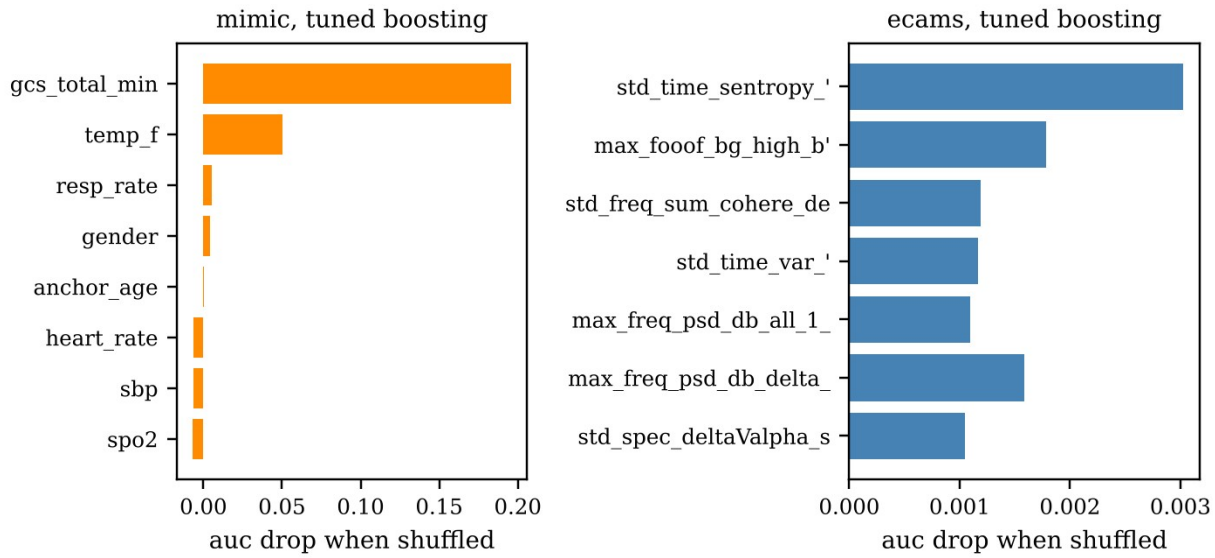


Figure 3. Permutation importance for the tuned boosting models on MIMIC (left) and ECAMS (right).

K-means and Silhouette Score (Depth Topic 2)

K-means groups patients by distance to cluster centers and never sees the delirium label, so it shows what structure the features hold on their own (Arthur & Vassilvitskii, 2007). The silhouette score compares each patient's average distance to its own cluster with the distance to the nearest other cluster, from negative one to one (Rousseeuw, 1987), and a common rule of

thumb says values under about 0.25 mean no real structure. I ran k from 2 to 10 with 20 random starts on scaled and unscaled MIMIC, scaled and unscaled ECAMS, and ECAMS reduced to 10 principal components. Figure 4 shows that the elbow plot has no elbow in any space, since inertia falls smoothly. Silhouette is low everywhere for the scaled data, 0.12 to 0.17 for MIMIC, 0.07 to 0.13 for ECAMS, and 0.13 to 0.18 for the PCA space, with the best k at 9 for MIMIC (0.171) and 2 for both ECAMS versions.

The unscaled scores are the trap. Unscaled ECAMS scored 0.966, but that solution is one cluster of 372 patients and one patient alone, pulled away by a few EEG power ratio features with average spreads above 3,200. Silhouette will happily reward a degenerate or scale-driven solution. Clustering has no test set to overfit, so I checked stability instead. A single random start was unreliable, since ten single-start runs on MIMIC at k of 2 agreed at an adjusted Rand index (ARI) of only 0.18 on average, against 0.94 on ECAMS, which is why I used 20 starts and treat the MIMIC clusters as fragile.

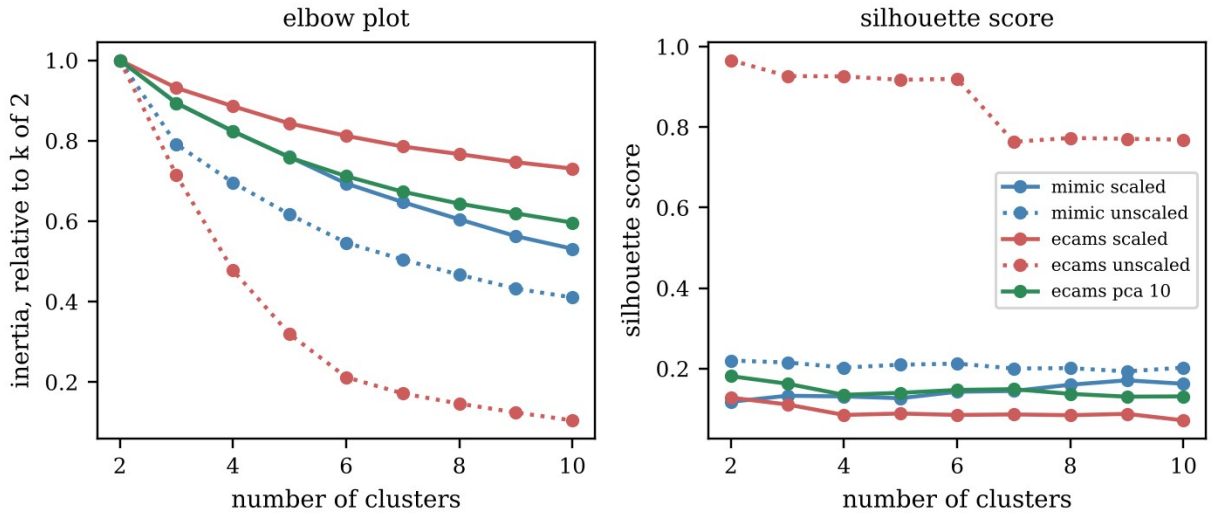


Figure 4. Elbow plot (left) and silhouette score (right) for K-means in five feature spaces. Dotted lines are unscaled.

Even with weak silhouettes, the ECAMS clusters mean something. The two-cluster solution split 184 patients with a delirium rate of 92 percent from 189 with 44 percent, from features that never saw the label (ARI 0.22), and the 10-component version gave the same split. Figure 5 shows why the score stays low, since the clusters look like two halves of one cloud. On MIMIC, the three clusters (41, 44, and 55 stays) separate patients with very low GCS scores (average lowest GCS of 3.7, 86 percent male, 43 percent delirious), hypertensive patients (24 percent), and older hypotensive patients (29 percent), which is sensible clinically, but the delirium rates only differ by about 20 points in groups of 40 to 55 stays, and at four clusters the ARI with delirium is zero.

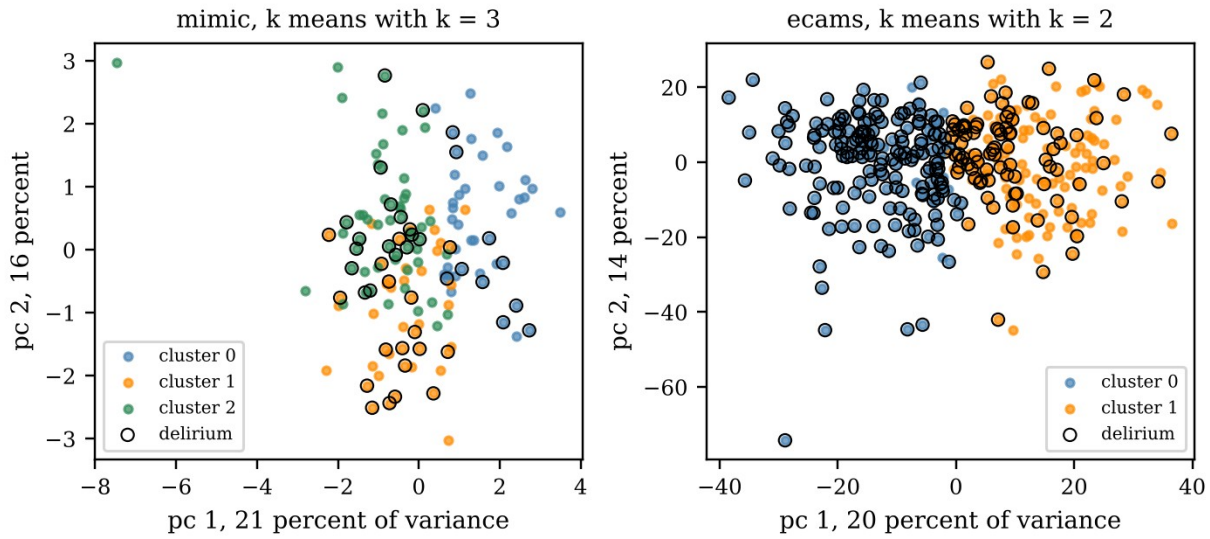


Figure 5. K-means clusters in the first two principal components for MIMIC at k = 3 (left) and ECAMS at k = 2 (right). Circled points are delirious.

DBSCAN and Hierarchical Clustering (Breadth)

DBSCAN labels points as dense core regions or noise using an eps radius and a min_samples count (Ester et al., 1996). None of my k-distance curves had a clean knee, so I searched a grid. On both datasets, every setting either put almost all points in one cluster or made most of them noise. On MIMIC, eps of 2.0 with min_samples of 5 gave 3 clusters with 44

percent noise and a silhouette of 0.18, and eps of 2.25 or more gave a single cluster. On scaled ECAMS, eps of 22 gave 3 small clusters with 88 percent noise, and eps of 25 or more gave one. The noise was informative on ECAMS, though. At eps of 34, about 9 percent of patients were noise, and 79 percent of them were delirious, compared with 67 percent of the rest, while on MIMIC the noise was not enriched (22 to 35 percent against 32 to 35 percent). A result that flips from one big cluster to mostly noise with a small change in eps is the clustering version of overfitting, because the solution depends on a tuning choice more than on the data.

For hierarchical clustering, the linkage decided everything. Complete, average, and single linkage chain, so at three clusters the largest cluster held 96.5 to 99.5 percent of ECAMS patients and 98.6 percent of MIMIC stays. Their silhouettes (up to 0.48) and cophenetic correlations (0.82 for the average on ECAMS) were the highest, but only because everything sat in one cluster. Ward linkage (Ward, 1963) was the only one with balanced clusters, and its silhouette was the lowest (0.095 on ECAMS). The ECAMS dendrogram in Figure 6 has two clear branches near a Ward distance of 290, and the four Ward clusters had delirium rates of 85, 98, 45, and 69 percent, while the MIMIC tree has no obvious cut and its rates were 27, 42, and 31 percent, plus one stay on its own.

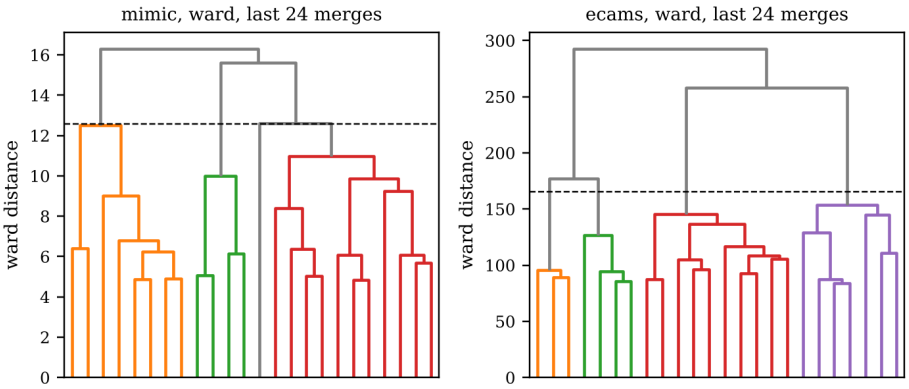


Figure 6. Ward linkage dendrograms for MIMIC (left) and ECAMS (right), cut at four clusters.

Expected and Unexpected Results

Some things went the way the EDA said. Scaling mattered, ECAMS was easier than MIMIC, and a few components held most of ECAMS. Other results surprised me. I expected boosting to beat the random forest, and it did not on either dataset. Tuning added about 0.02 AUC for KNN and boosting on MIMIC, 0.04 for KNN on ECAMS, and nothing for boosting on ECAMS. The 0.918 MIMIC holdout AUC looked like a win until I compared it with the 0.759 CV score. The silhouette results surprised me most, because the highest scores (0.97 for unscaled ECAMS, near 0.5 for chained linkages) belonged to the worst clusterings, and some of the most useful splits, K-means on ECAMS and Ward linkage, had low silhouettes.

Conclusions

Table 1 pulls the main numbers together, and the two datasets agree on the main points. On ECAMS, every reasonable supervised model lands between 0.87 and 0.90 AUC; the choice of model matters little, and the random forest is best at CV 0.898. On MIMIC, the models spread out more (CV 0.74 to 0.78), and the best is again the random forest, at 0.779, with GCS doing almost all of the work. Overfitting is the main risk, because training AUC reaches 1.0 for KNN at k of 1 and for boosting by round 300, while MIMIC validation AUC peaks near 0.78 early. A larger k shrinks the gap, and for boosting early stopping, a slower learning rate and feature subsampling helped most. The unsupervised results say that scaling is required, that silhouette and cophenetic correlation can favor useless solutions, and that the structure which matters for delirium is in small or extreme groups, such as ECAMS noise points and the Ward cluster at 98 percent, and not in well-separated clusters. The limits are real. MIMIC is a demo subset with repeated patients and 9 positive test stays; ECAMS comes from one hospital, and I did not tune the decision threshold. Next, I would use the full MIMIC IV database with patient-level splits.

References

Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. Proceedings of the 18th Annual ACM SIAM Symposium on Discrete Algorithms (pp. 1027 to 1035).

Ester, M., Kriegel, H. P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (pp. 226 to 231).

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189 to 1232.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning (2nd ed.). Springer.

Johnson, A. E. W., Bulgarelli, L., Shen, L., Gayles, A., Shammout, A., Horng, S., Pollard, T. J., Hao, S., Moody, B., Gow, B., Lehman, L. H., Celi, L. A., & Mark, R. G. (2023a). MIMIC IV, a freely accessible electronic health record dataset. Scientific Data, 10, 1. https://doi.org/10.1038/s41597-022-01899-x

Johnson, A., Bulgarelli, L., Pollard, T., Horng, S., Celi, L. A., & Mark, R. (2023b). MIMIC IV Clinical Database Demo (version 2.2). PhysioNet. https://doi.org/10.13026/07hj-2a80

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825 to 2830.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53 to 65.

Scikit-learn developers. (n.d.). Clustering and Ensemble methods user guides. https://scikit-learn.org/stable/modules/clustering.html and https://scikit-learn.org/stable/modules/ensemble.html

van Sleuwen, M., Sun, H., Eckhardt, C., et al. (2022). Physiological assessment of delirium severity: The electroencephalographic confusion assessment method severity score (E CAM S). Critical Care Medicine, 50(1), e11 to e19. https://doi.org/10.1097/CCM.0000000000005224

Ward, J. H. (1963). Hierarchical grouping to optimize an objective function. Journal of the American Statistical Association, 58(301), 236 to 244.

Appendix A: Use of AI

I used Claude (Sonnet 5.5) on this milestone. Claude helped with finding resources, double-checking references, grading against the provided rubric, and providing feedback. I used the provided feedback to make some modifications to my Milestone 2. I also used Grammarly to help with sentence structure, spelling mistakes, and grammatical errors. This project, dataset choices, and the cross-validation design carried over from my Milestone One notebooks.

Every number in this summary comes from the executed notebooks listed in Appendix B. Outside sources are cited in the References section. I used the scikit-learn user guides and the papers listed there to understand the methods.

Appendix B: Jupyter Notebooks

The executed notebooks for Weeks 8 to 11 follow this page: Week8_KNN, Week9_GradientBoost, Week10_KMeans, and Week11_DBSCAN_HAC.

GitHub repository: https://github.com/MHembree82/bu-capstone-milestone-two

Week 8 notebook, k nearest neighbors and distance metrics, Mike Hembree

Same project as the earlier weeks. The capstone is an early warning model for ICU delirium on MIMIC IV, and the MIMIC part of this notebook runs on the MIMIC IV Clinical Database Demo (100 patients, 140 ICU stays) because the full credentialed access is still pending. The second dataset is the ECAMS EEG data from Massachusetts General Hospital (373 patients, 1189 EEG features, delirium called by the CAM algorithm), the same two datasets as the Milestone One notebooks. This week is classification on both, a positive delirium screen for MIMIC (45 of 140 stays) and a delirious call for ECAMS (254 of 373 patients). Validation is repeated stratified five fold cross validation with three repeats, and imputation, scaling and any tuning happen inside each training fold. The MIMIC feature table is the same one from the Week 2 notebook and the first code cell saves it so the later weeks can load it.

In [1]:
```
# imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from sklearn.model_selection import train_test_split, RepeatedStratifiedKFold, cross_validate, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegressionCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, recall_score, precision_score, f1_score, accuracy_score

warnings.filterwarnings('ignore')
plt.rcParams['figure.dpi'] = 110
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 9
plt.rcParams['axes.titlesize'] = 9
plt.rcParams['axes.labelsize'] = 9
plt.rcParams['xtick.labelsize'] = 8
plt.rcParams['ytick.labelsize'] = 8
plt.rcParams['legend.fontsize'] = 7.5
```

In [2]:
```
# build the mimic feature table from the raw demo tables, same item ids as the Week 2 notebook
icu = pd.read_csv('mimic-iv-clinical-database-demo-2.2/icu/icustays.csv.gz')
pts = pd.read_csv('mimic-iv-clinical-database-demo-2.2/hosp/patients.csv.gz')
ce = pd.read_csv('mimic-iv-clinical-database-demo-2.2/icu/chartevents.csv.gz', low_memory = False)

vitals = {220045: 'heart_rate', 220179: 'sbp', 220180: 'dbp', 220210: 'resp_rate', 223761: 'temp_f', 220277: 'spo2'
gcs_items = {223900: 'gcs_verbal', 223901: 'gcs_motor', 220739: 'gcs_eye'}

feat = icu[['subject_id', 'hadm_id', 'stay_id', 'first_careunit', 'los']].copy()
feat = feat.merge(pts[['subject_id', 'gender', 'anchor_age']], on = 'subject_id', how = 'left')
for iid, name in vitals.items():
    stay_avg = ce[ce.itemid == iid].groupby('stay_id')['valuenum'].mean().rename(name)
    feat = feat.merge(stay_avg, on = 'stay_id', how = 'left')
for iid, name in gcs_items.items():
    stay_min = ce[ce.itemid == iid].groupby('stay_id')['valuenum'].min().rename(name)
    feat = feat.merge(stay_min, on = 'stay_id', how = 'left')
feat['gcs_total_min'] = feat['gcs_verbal'] + feat['gcs_motor'] + feat['gcs_eye']
delir = ce[ce.itemid == 228332]
delir_flag = delir.groupby('stay_id')['value'].apply(lambda v: int((v == 'Positive').any())).rename('delirium_posit
feat = feat.merge(delir_flag, on = 'stay_id', how = 'left')
feat['delirium_positive'] = feat['delirium_positive'].fillna(0).astype(int)
feat['gender'] = (feat['gender'] == 'M').astype(int)
feat.to_csv('delirium_features.csv', index = False)
print(feat.shape, feat['delirium_positive'].sum(), 'positive stays', feat['subject_id'].nunique(), 'patients')
```

```
(140, 18) 45 positive stays 100 patients
```

In [3]:
```
# load both datasets, mimic keeps its missing values and gets imputed inside the pipelines
mcols = ['anchor_age', 'gender', 'heart_rate', 'sbp', 'dbp', 'resp_rate', 'temp_f', 'spo2', 'gcs_total_min']
Xm = feat[mcols]
ym = feat['delirium_positive']

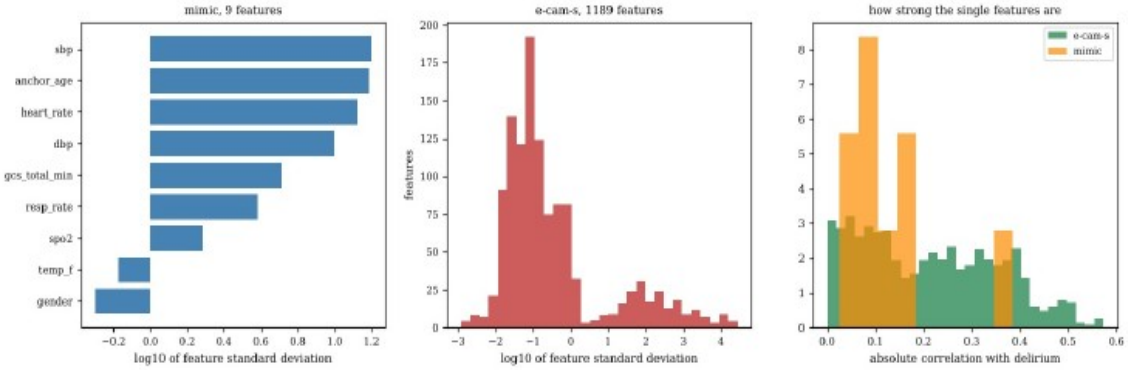
Xe = pd.read_csv('ecams/features_alldata.csv')
clin = pd.read_csv('ecams/E_CAM_S_Data.csv').sort_values('SubjectID-used_for_feature_extraction').reset_index(drop
ye = clin['CAM algorithm (CAM 1 & 2 , and 3 or 4 SATISFIED - DELIRIUM POSITIVE)'].astype(int)

data = {'mimic': (Xm, ym), 'ecams': (Xe, ye)}
cv_clf = RepeatedStratifiedKFold(n_splits = 5, n_repeats = 3, random_state = 1)
splits = {k: train_test_split(X, y, test_size = 0.2, random_state = 42, stratify = y) for k, (X, y) in data.items()
```

```
for k, (X, y) in data.items():
    print(k, X.shape, 'positive rate', round(y.mean(), 3), 'missing values', int(X.isna().sum().sum()))
```

```
mimic (140, 9) positive rate 0.321 missing values 4
ecams (373, 1189) positive rate 0.681 missing values 0
```

In [4]:
```
# eda, how different the feature scales are and how strongly each feature relates to the outcome
fig, ax = plt.subplots(1, 3, figsize = (13, 4))
sm = Xm.std().sort_values()
ax[0].barh(sm.index, np.log10(sm.values), color = 'steelblue')
ax[0].set_xlabel('log10 of feature standard deviation')
ax[0].set_title('mimic, 9 features')
ax[1].hist(np.log10(Xe.std()), bins = 30, color = 'indianred')
ax[1].set_xlabel('log10 of feature standard deviation')
ax[1].set_ylabel('features')
ax[1].set_title('e-cam-s, 1189 features')
cm = Xe.corrwith(ye).abs()
cmm = Xm.fillna(Xm.median()).corrwith(ym).abs()
ax[2].hist(cm, bins = 30, color = 'seagreen', alpha = 0.8, density = True, label = 'e-cam-s')
ax[2].hist(cmm, bins = 9, color = 'darkorange', alpha = 0.7, density = True, label = 'mimic')
ax[2].set_xlabel('absolute correlation with delirium')
ax[2].set_title('how strong the single features are')
ax[2].legend()
plt.tight_layout()
plt.savefig('figs/w8_eda.png', dpi = 200)
plt.show()
print('mimic std range:', round(Xm.std().min(), 2), 'to', round(Xm.std().max(), 2))
print('ecams std range:', round(Xe.std().min(), 4), 'to', round(Xe.std().max(), 1))
print('mimic top correlations:', cmm.sort_values(ascending = False).head(3).round(2).to_dict())
print('ecams features over 0.3:', int((cm > 0.3).sum()), 'max:', round(cm.max(), 2))
cmat = Xe.corr().abs().values.copy()
np.fill_diagonal(cmat, 0)
print('ecams features with a partner over 0.95:', int((cmat.max(axis = 1) > 0.95).sum()))
```




```
mimic std range: 0.5 to 15.85
ecams std range: 0.0012 to 27354.8
mimic top correlations: {'gcs_total_min': 0.38, 'spo2': 0.17, 'temp_f': 0.15}
ecams features over 0.3: 358 max: 0.57
ecams features with a partner over 0.95: 1058
```

The scales are the first thing to deal with. On MIMIC the standard deviations run from about 0.5 for the gender flag to almost 16 for systolic pressure, so systolic pressure, age and heart rate will dominate a raw distance. On ECAMS it's far worse, the standard deviations go from 0.0012 to over 27,000. The ECAMS features are also very redundant, 1058 of the 1189 have another feature correlated with them above 0.95, and 358 correlate with delirium at more than 0.3 (the largest is 0.57). On MIMIC the strongest single feature is gcs_total_min at 0.38 and the rest are weak, spo2 at 0.17 and temp_f at 0.15. So I expect scaling to matter a lot for KNN, and I expect ECAMS to do better than MIMIC because it has more rows and stronger single features.

In [5]:
```
# knn helper, imputation and scaling are inside the pipeline so every fold stays clean
def knn_pipe(k, metric = 'euclidean', scale = True, weights = 'uniform'):
    steps = [SimpleImputer(strategy = 'median')]
    if scale:
        steps.append(StandardScaler())
    algo = 'brute' if metric == 'cosine' else 'auto'
    steps.append(KNeighborsClassifier(n_neighbors = k, metric = metric, weights = weights, algorithm = algo))
    return make_pipeline(*steps)

def cv_auc(name, model, train = False):
    X, y = data[name]
    r = cross_validate(model, X, y, cv = cv_clf, scoring = 'roc_auc', return_train_score = train, n_jobs = 2)
    if train:
```

```
        return r['train_score'].mean(), r['test_score'].mean(), r['test_score'].std()
    return r['test_score'].mean(), r['test_score'].std()

ks = {'mimic': [1, 3, 5, 7, 11, 15, 21, 31, 45, 61], 'ecams': [1, 3, 5, 7, 11, 15, 21, 31, 45, 61, 91]}
metrics = ['euclidean', 'manhattan', 'chebyshev', 'cosine']
rows = []
for name in data:
    for m in metrics:
        for k in ks[name]:
            tr, va, sd = cv_auc(name, knn_pipe(k, m), train = True)
            rows.append({'data': name, 'metric': m, 'k': k, 'train auc': tr, 'cv auc': va, 'sd': sd})
sweep = pd.DataFrame(rows)
sweep.to_csv('figs/w8_knn_sweep.csv', index = False)
best = sweep.loc[sweep.groupby(['data', 'metric'])['cv auc'].idxmax()]
best.round(3)
```

Out[5]:

	data	metric	k	train auc	cv auc	sd
68	ecams	chebyshev	21	0.817	0.750	0.065
80	ecams	cosine	31	0.897	0.884	0.038
48	ecams	euclidean	45	0.889	0.879	0.038
59	ecams	manhattan	45	0.892	0.883	0.037
23	mimic	chebyshev	7	0.827	0.694	0.083
37	mimic	cosine	31	0.795	0.759	0.073
5	mimic	euclidean	15	0.809	0.730	0.056
17	mimic	manhattan	31	0.799	0.767	0.092

With scaled features the metric matters on MIMIC and mostly doesn't on ECAMS. On MIMIC manhattan is best at a cv auc of 0.767 (k of 31), then cosine at 0.759, euclidean at 0.730 and chebyshev last at 0.694. On ECAMS cosine, manhattan and euclidean land within 0.005 of each other (0.884, 0.883 and 0.879) and chebyshev falls way behind at 0.750. Chebyshev only looks at the single biggest coordinate difference, which with 1189 features throws away almost everything. My guess on why manhattan likes the small MIMIC data is that squared differences let one odd vital sign pull a stay far from everything else, while absolute differences are gentler.

In [6]:
```
# unscaled against scaled, same grid with the metric fixed at euclidean
rows = []
for name in data:
    for sc in [False, True]:
        res = [(k, cv_auc(name, knn_pipe(k, 'euclidean', scale = sc))[0]) for k in ks[name]]
        kb, ab = max(res, key = lambda t: t[1])
        rows.append({'data': name, 'scaled': sc, 'best k': kb, 'cv auc': ab})
pd.DataFrame(rows).round(3)
```

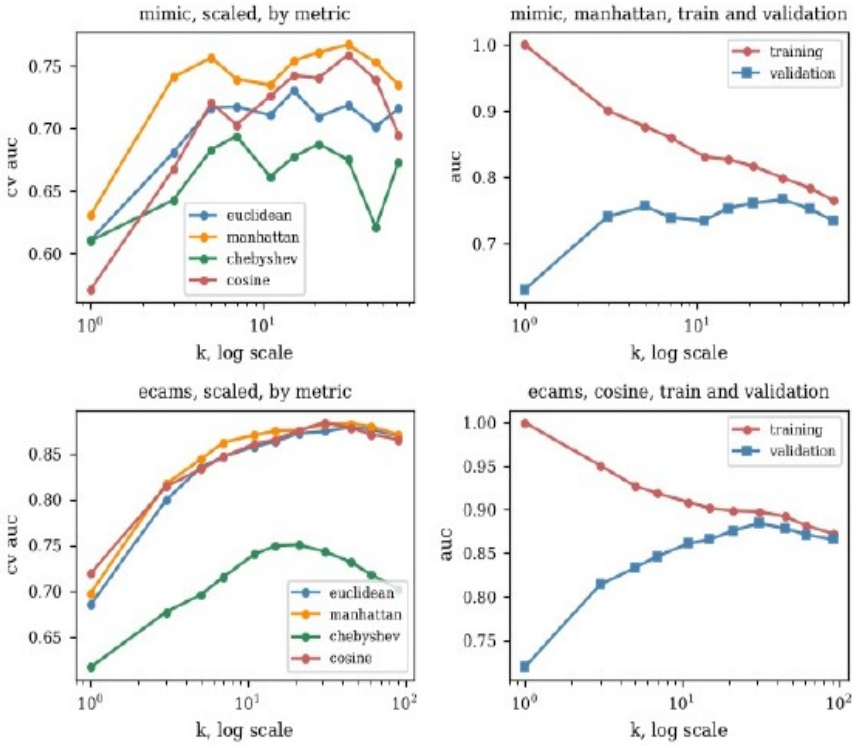
Out[6]:

	data	scaled	best k	cv auc
0	mimic	False	3	0.643
1	mimic	True	15	0.730
2	ecams	False	91	0.698
3	ecams	True	45	0.879

Scaling is the biggest single effect in the notebook. On MIMIC the best unscaled model gets a cv auc of 0.643 and the scaled one 0.730, and on ECAMS it's 0.698 unscaled against 0.879 scaled. Without scaling a handful of features with huge variances are the only ones that count in the distance, which matches what the EDA showed.

In [7]:
```
# figure, metric curves on the left and training against validation on the right
fig, ax = plt.subplots(2, 2, figsize = (6.5, 5.2))
cols = {'euclidean': 'steelblue', 'manhattan': 'darkorange', 'chebyshev': 'seagreen', 'cosine': 'indianred'}
for i, name in enumerate(data):
    s = sweep[sweep.data == name]
    for m in metrics:
        t = s[s.metric == m]
        ax[i, 0].plot(t['k'], t['cv auc'], 'o-', color = cols[m], label = m, markersize = 3.5)
    ax[i, 0].set_xscale('log')
    ax[i, 0].set_xlabel('k, log scale')
    ax[i, 0].set_ylabel('cv auc')
    ax[i, 0].set_title(name + ', scaled, by metric')
```

```
    ax[i, 0].legend(fontsize = 7)
    bm = best[best.data == name].sort_values('cv auc').iloc[-1]['metric']
    t = s[s.metric == bm]
    ax[i, 1].plot(t['k'], t['train auc'], 'o-', color = 'indianred', label = 'training', markersize = 3.5)
    ax[i, 1].plot(t['k'], t['cv auc'], 's-', color = 'steelblue', label = 'validation', markersize = 3.5)
    ax[i, 1].set_xscale('log')
    ax[i, 1].set_xlabel('k, log scale')
    ax[i, 1].set_ylabel('auc')
    ax[i, 1].set_title(name + ', ' + bm + ', train and validation')
    ax[i, 1].legend(fontsize = 7)
plt.tight_layout()
plt.savefig('figs/w8_knn.png', dpi = 220)
plt.show()
```



The right column is the overfitting picture. At k of 1 the training auc is exactly 1.0 because every point is its own nearest neighbor, while validation is only 0.63 on MIMIC (manhattan) and 0.72 on ECAMS (cosine). Training and validation come together as k grows. At k of 31 the gap is 0.03 on MIMIC (0.799 against 0.767) and 0.013 on ECAMS (0.897 against 0.884). The MIMIC validation curve is jumpy, it bounces between 0.735 and 0.767 from k of 11 to 45, because each training fold only has 112 stays, so I wouldn't read anything into the exact peak. The ECAMS curve is smooth and flattens out around k of 30 to 45.

In [8]:
```
# pca before knn on the e-cam-s features, the 1189 columns are mostly copies of each other
rows = []
for nc in [2, 5, 10, 20, 40, 80]:
    for k in [11, 21, 31]:
        m = make_pipeline(StandardScaler(), PCA(nc), KNeighborsClassifier(n_neighbors = k))
        rows.append({'components': nc, 'k': k, 'cv auc': cv_auc('ecams', m)[0]})
pca_knn = pd.DataFrame(rows).pivot(index = 'components', columns = 'k', values = 'cv auc')
pca_knn.round(3)
```

Out[8]:

components	k	11	21	31
	2	0.811	0.833	0.843
	5	0.829	0.847	0.859
	10	0.853	0.871	0.873
	20	0.855	0.870	0.873
	40	0.859	0.870	0.873
	80	0.859	0.871	0.876

Ten principal components already gets a cv auc of 0.873 at k of 31, and 80 components only gets 0.876, so the 1189 columns really are mostly copies of each other. Two components alone still reach 0.843. PCA didn't beat the full scaled data (about 0.88 at best) but it matches it with 10 columns instead of 1189.

In [9]:

```
# nested tuning, k metric and weights picked by an inner 5 fold search inside each outer fold
grid = {'kneighborsclassifier__n_neighbors': [3, 5, 9, 15, 21, 31, 45], 'kneighborsclassifier__metric': ['euclidean
rf_grid = {'max_depth': [3, 5, None], 'min_samples_leaf': [1, 3, 5], 'max_features': ['sqrt', 0.5]}
models = {
    'knn, k 5 default': knn_pipe(5),
    'knn, tuned inside folds': GridSearchCV(knn_pipe(5), grid, scoring = 'roc_auc', cv = 5),
    'logistic, cv regularized': make_pipeline(SimpleImputer(strategy = 'median'), StandardScaler(), LogisticRegress
    'random forest, default': make_pipeline(SimpleImputer(strategy = 'median'), RandomForestClassifier(n_estimators
}
scoring = ('roc_auc', 'recall', 'precision', 'f1', 'accuracy')
rows = []
for name in data:
    X, y = data[name]
    for mn, m in models.items():
        r = cross_validate(m, X, y, cv = cv_clf, scoring = scoring, n_jobs = 2)
        rows.append({'data': name, 'model': mn, 'auc': r['test_roc_auc'].mean(), 'auc sd': r['test_roc_auc'].std(),
        print(name, mn, round(rows[-1]['auc'], 3))
knn_tab = pd.DataFrame(rows)
knn_tab.to_csv('figs/w8_summary.csv', index = False)
knn_tab.round(3)
```

```
mimic knn, k 5 default 0.717
mimic knn, tuned inside folds 0.741
mimic logistic, cv regularized 0.75
mimic random forest, default 0.779
ecams knn, k 5 default 0.835
ecams knn, tuned inside folds 0.874
ecams logistic, cv regularized 0.876
ecams random forest, default 0.898
```

Out[9]:

	data	model	auc	auc sd	recall	precision	f1	accuracy
0	mimic	knn, k 5 default	0.717	0.088	0.563	0.577	0.557	0.721
1	mimic	knn, tuned inside folds	0.741	0.085	0.178	0.586	0.248	0.698
2	mimic	logistic, cv regularized	0.750	0.088	0.311	0.529	0.365	0.707
3	mimic	random forest, default	0.779	0.105	0.519	0.653	0.565	0.745
4	ecams	knn, k 5 default	0.835	0.033	0.828	0.848	0.836	0.780
5	ecams	knn, tuned inside folds	0.874	0.038	0.867	0.851	0.857	0.805
6	ecams	logistic, cv regularized	0.876	0.039	0.889	0.847	0.867	0.814
7	ecams	random forest, default	0.898	0.024	0.882	0.858	0.869	0.819

Under nested tuning KNN is competitive on ECAMS and behind on MIMIC. On ECAMS the tuned KNN gets an auc of 0.874, the same as cv regularized logistic regression (0.876) and a bit below the default random forest (0.898). On MIMIC the order is forest 0.779, logistic 0.750, tuned KNN 0.741 and k of 5 KNN 0.717. Tuning k and the metric inside the folds bought about 0.02 on MIMIC and 0.04 on ECAMS over a default k of 5. The tuned MIMIC KNN has a recall of only 0.18, because it settled on a big k with distance weights, and with 32 percent positives a large neighborhood mostly votes negative at a cutoff of 0.5. Auc is fine with that but recall isn't, so for a screening tool the threshold would have to move.

In [10]:

```
# holdout check with a bootstrap interval on the test auc
rng = np.random.default_rng(0)
```

```
rows = []
for name in data:
    Xtr, Xte, ytr, yte = splits[name]
    for mn in ['knn, tuned inside folds', 'logistic, cv regularized', 'random forest, default']:
        m = models[mn].fit(Xtr, ytr)
        p = m.predict_proba(Xte)[:, 1]
        yt = yte.values
        boots = []
        for b in range(1000):
            idx = rng.integers(0, len(yt), len(yt))
            if yt[idx].min() != yt[idx].max():
                boots.append(roc_auc_score(yt[idx], p[idx]))
        rows.append({'data': name, 'model': mn, 'test rows': len(yt), 'positives': int(yt.sum()), 'test auc': roc_a
        if 'knn' in mn:
            print(name, m.best_params_ if hasattr(m, 'best_params_') else '')
pd.DataFrame(rows).round(3)
```

```
mimic {'kneighborsclassifier__metric': 'manhattan', 'kneighborsclassifier__n_neighbors': 31, 'kneighborsclassifier__
weights': 'distance'}
ecams {'kneighborsclassifier__metric': 'manhattan', 'kneighborsclassifier__n_neighbors': 45, 'kneighborsclassifier__
weights': 'uniform'}
```

Out[10]:

	data	model	test rows	positives	test auc	low	high
0	mimic	knn, tuned inside folds	28	9	0.836	0.667	0.967
1	mimic	logistic, cv regularized	28	9	0.801	0.614	0.946
2	mimic	random forest, default	28	9	0.854	0.678	0.994
3	ecams	knn, tuned inside folds	75	51	0.868	0.784	0.940
4	ecams	logistic, cv regularized	75	51	0.892	0.804	0.955
5	ecams	random forest, default	75	51	0.873	0.781	0.943

On the holdout split the tuned KNN gets an auc of 0.836 on MIMIC and 0.868 on ECAMS. The intervals are wide though, 0.67 to 0.97 on MIMIC where the 28 test stays only include 9 positives, and 0.78 to 0.94 on ECAMS. The MIMIC holdout numbers (0.80 to 0.85 for all three models) are higher than their cv numbers (0.74 to 0.78), which I read as the small test set being friendly, not the models being better than cv said. The search picked manhattan on both datasets, with k of 31 and distance weights on MIMIC and k of 45 on ECAMS.

To sum up, KNN works about as well as logistic regression on ECAMS and a bit worse on MIMIC, scaling is not optional, and the metric only matters when the feature count is small or the metric ignores most of the features (chebyshev).

Week 9 notebook, gradient boosting, Mike Hembree

Same project as the earlier weeks. The capstone is an early warning model for ICU delirium on MIMIC IV, and the MIMIC part of this notebook runs on the MIMIC IV Clinical Database Demo (100 patients, 140 ICU stays) because the full credentialed access is still pending. The second dataset is the ECAMS EEG data from Massachusetts General Hospital (373 patients, 1189 EEG features, delirium called by the CAM algorithm), the same two datasets as the Milestone One notebooks. Same validation as Week 8, repeated stratified five fold cross validation with three repeats, and anything tuned is tuned inside each training fold. Gradient boosting builds small trees one after another, each one fit to the mistakes of the trees before it, and the knobs I care about are the learning rate, the number of rounds, tree depth, and the regularization options sklearn has (row subsampling, minimum leaf size, feature subsampling, and early stopping). All fits use max_features of sqrt unless noted, which keeps the 1189 column ECAMS runs fast.

In [1]:
```
# imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from sklearn.model_selection import train_test_split, RepeatedStratifiedKFold, StratifiedKFold, cross_validate, Ran
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegressionCV
from sklearn.inspection import permutation_importance
from sklearn.metrics import roc_auc_score, recall_score, precision_score, f1_score, accuracy_score

warnings.filterwarnings('ignore')
plt.rcParams['figure.dpi'] = 110
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 9
plt.rcParams['axes.titlesize'] = 9
plt.rcParams['axes.labelsize'] = 9
plt.rcParams['xtick.labelsize'] = 8
plt.rcParams['ytick.labelsize'] = 8
plt.rcParams['legend.fontsize'] = 7.5
```

In [2]:
```
# load both datasets, the mimic feature table was saved by the Week 8 notebook
feat = pd.read_csv('delirium_features.csv')
mcols = ['anchor_age', 'gender', 'heart_rate', 'sbp', 'dbp', 'resp_rate', 'temp_f', 'spo2', 'gcs_total_min']
Xm = feat[mcols]
ym = feat['delirium_positive']

Xe = pd.read_csv('ecams/features_alldata.csv')
clin = pd.read_csv('ecams/E_CAM_S_Data.csv').sort_values('SubjectID-used_for_feature_extraction').reset_index(drop
ye = clin['CAM algorithm (CAM 1 & 2 , and 3 or 4 SATISFIED - DELIRIUM POSITIVE)'].astype(int)

data = {'mimic': (Xm, ym), 'ecams': (Xe, ye)}
cv_clf = RepeatedStratifiedKFold(n_splits = 5, n_repeats = 3, random_state = 1)
splits = {k: train_test_split(X, y, test_size = 0.2, random_state = 42, stratify = y) for k, (X, y) in data.items()
for k, (X, y) in data.items():
    print(k, X.shape, 'positive rate', round(y.mean(), 3))
```

```
mimic (140, 9) positive rate 0.321
ecams (373, 1189) positive rate 0.681
```

In [3]:
```
# gradient boosting helper, a staged cv curve gives train and validation auc after every tree
def gb_model(**kw):
    base = dict(n_estimators = 300, learning_rate = 0.1, max_depth = 2, random_state = 0)
    if 'max_features' not in kw:
        kw['max_features'] = 'sqrt'
    base.update(kw)
    return make_pipeline(SimpleImputer(strategy = 'median'), GradientBoostingClassifier(**base))

def staged_cv(name, n_splits = 5, n_repeats = 3, **kw):
    X, y = data[name]
    cv = RepeatedStratifiedKFold(n_splits = n_splits, n_repeats = n_repeats, random_state = 1)
    tr_curves, va_curves = [], []
    for a, b in cv.split(X, y):
        m = gb_model(**kw).fit(X.iloc[a], y.iloc[a])
        imp = m[0]
        Xa = imp.transform(X.iloc[a])
        Xb = imp.transform(X.iloc[b])
        tr_curves.append([roc_auc_score(y.iloc[a], p[:, 1]) for p in m[1].staged_predict_proba(Xa)])
        va_curves.append([roc_auc_score(y.iloc[b], p[:, 1]) for p in m[1].staged_predict_proba(Xb)])
```

```
    return np.mean(tr_curves, axis = 0), np.mean(va_curves, axis = 0)

curves = {}
for name in data:
    for lr in [0.01, 0.03, 0.1, 0.3]:
        curves[(name, 'lr', lr)] = staged_cv(name, learning_rate = lr, max_depth = 2)
    for d in [1, 2, 3, 5]:
        curves[(name, 'depth', d)] = staged_cv(name, learning_rate = 0.1, max_depth = d)
rows = []
for (name, kind, v), (tr, va) in curves.items():
    rows.append({'data': name, 'varied': kind, 'value': v, 'best round': int(va.argmax()) + 1, 'best val auc': va.m
stage_tab = pd.DataFrame(rows)
stage_tab.to_csv('figs/w9_stage_summary.csv', index = False)
stage_tab.round(3)
```

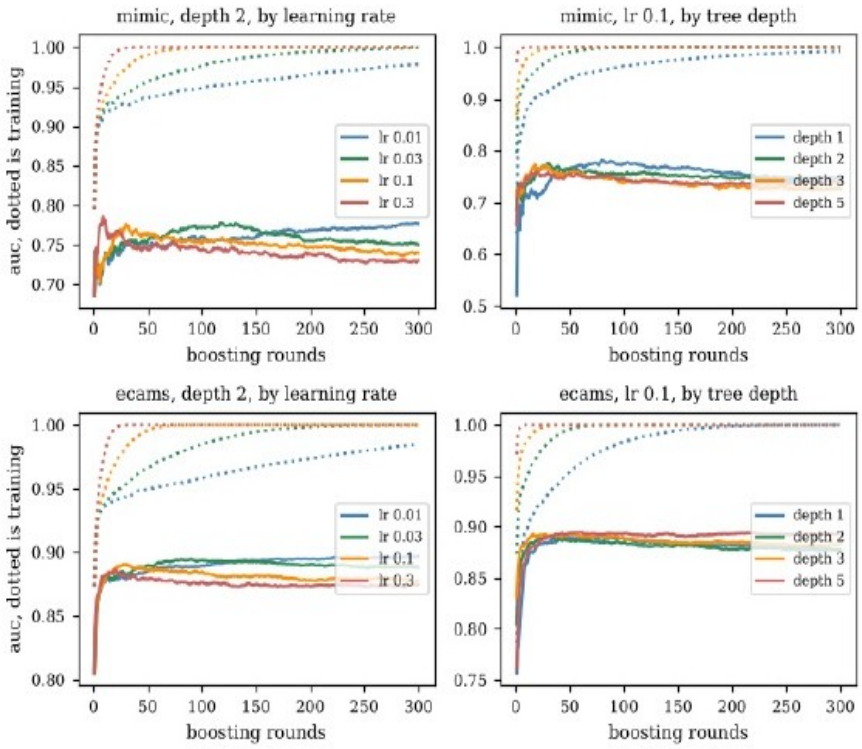
Out[3]:

	data	varied	value	best round	best val auc	val auc at 300	train auc at 300
0	mimic	lr	0.01	297	0.777	0.777	0.978
1	mimic	lr	0.03	118	0.778	0.750	0.999
2	mimic	lr	0.10	30	0.776	0.740	1.000
3	mimic	lr	0.30	9	0.786	0.730	1.000
4	mimic	depth	1.00	80	0.781	0.748	0.992
5	mimic	depth	2.00	30	0.776	0.740	1.000
6	mimic	depth	3.00	18	0.773	0.726	1.000
7	mimic	depth	5.00	19	0.759	0.741	1.000
8	ecams	lr	0.01	300	0.897	0.897	0.985
9	ecams	lr	0.03	90	0.895	0.888	1.000
10	ecams	lr	0.10	30	0.890	0.877	1.000
11	ecams	lr	0.30	21	0.886	0.874	1.000
12	ecams	depth	1.00	47	0.890	0.877	1.000
13	ecams	depth	2.00	30	0.890	0.877	1.000
14	ecams	depth	3.00	51	0.893	0.883	1.000
15	ecams	depth	5.00	55	0.895	0.893	1.000

Training auc reaches 1.0 or very close in every setting except the slowest learning rate, so what matters is the validation column. On MIMIC the validation auc peaks early and then slides down as more rounds are added. Learning rate 0.3 peaks at round 9 (0.786) and is at 0.730 by round 300, 0.1 peaks at round 30 and ends at 0.740, 0.03 peaks around round 118 and ends at 0.750, and only 0.01 is still creeping up at round 297 (0.777). Smaller learning rates give a higher number at 300 rounds, but the best validation auc is about 0.78 for every one of them, so the learning rate mostly changes how many rounds you can run before the model starts memorizing. Depth showed the same thing, depth 1 reached 0.781 at round 80, while depth 5 peaked at 0.759. ECAMS is much flatter, all settings sit between 0.886 and 0.897, the slowest learning rate is best at 0.897 and the fastest drifts down from 0.886 to 0.874. Deeper trees were a little better on ECAMS (depth 5 holds 0.893 at 300 rounds where depth 1 falls to 0.877), which is the opposite of MIMIC and probably reflects ECAMS having more rows and many redundant features to choose from.

In [4]:
```
# figure, validation auc by boosting round for four learning rates and four depths
fig, ax = plt.subplots(2, 2, figsize = (6.5, 5.2))
for i, name in enumerate(data):
    for lr, c in zip([0.01, 0.03, 0.1, 0.3], ['steelblue', 'seagreen', 'darkorange', 'indianred']):
        tr, va = curves[(name, 'lr', lr)]
        ax[i, 0].plot(np.arange(1, 301), va, color = c, label = 'lr ' + str(lr))
        ax[i, 0].plot(np.arange(1, 301), tr, color = c, linestyle = ':')
    ax[i, 0].set_xlabel('boosting rounds')
    ax[i, 0].set_ylabel('auc, dotted is training')
    ax[i, 0].set_title(name + ', depth 2, by learning rate')
    ax[i, 0].legend(fontsize = 7, loc = 'center right')
    for d, c in zip([1, 2, 3, 5], ['steelblue', 'seagreen', 'darkorange', 'indianred']):
        tr, va = curves[(name, 'depth', d)]
        ax[i, 1].plot(np.arange(1, 301), va, color = c, label = 'depth ' + str(d))
        ax[i, 1].plot(np.arange(1, 301), tr, color = c, linestyle = ':')
    ax[i, 1].set_xlabel('boosting rounds')
    ax[i, 1].set_title(name + ', lr 0.1, by tree depth')
    ax[i, 1].legend(fontsize = 7, loc = 'center right')
plt.tight_layout()
```

```
plt.savefig('figs/w9_staged.png', dpi = 220)
plt.show()
```



In [5]:
```
# regularization knobs one at a time at learning rate 0.1, depth 3, 150 rounds
settings = {
    'plain': dict(),
    'subsample 0.5': dict(subsample = 0.5),
    'min leaf 5': dict(min_samples_leaf = 5),
    'min leaf 10': dict(min_samples_leaf = 10),
    'max features 0.05': dict(max_features = 0.05),
    'max features all': dict(max_features = None),
    'subsample 0.5, min leaf 5': dict(subsample = 0.5, min_samples_leaf = 5)
}
rows = []
for name in data:
    for sn, kw in settings.items():
        if name == 'mimic' and 'max features 0.05' in sn:
            continue
        r = cross_validate(gb_model(n_estimators = 150, learning_rate = 0.1, max_depth = 3, **kw), data[name][0], d
        rows.append({'data': name, 'setting': sn, 'train auc': r['train_score'].mean(), 'cv auc': r['test_score'].m
reg_tab = pd.DataFrame(rows)
reg_tab.to_csv('figs/w9_regularization.csv', index = False)
reg_tab.round(3)
```

Out[5]:

	data	setting	train auc	cv auc	gap
0	mimic	plain	1.0	0.735	0.265
1	mimic	subsample 0.5	1.0	0.748	0.252
2	mimic	min leaf 5	1.0	0.724	0.276
3	mimic	min leaf 10	1.0	0.741	0.259
4	mimic	max features all	1.0	0.730	0.270
5	mimic	subsample 0.5, min leaf 5	1.0	0.770	0.230
6	ecams	plain	1.0	0.884	0.116
7	ecams	subsample 0.5	1.0	0.886	0.114
8	ecams	min leaf 5	1.0	0.888	0.112
9	ecams	min leaf 10	1.0	0.888	0.112
10	ecams	max features 0.05	1.0	0.884	0.116
11	ecams	max features all	1.0	0.867	0.133
12	ecams	subsample 0.5, min leaf 5	1.0	0.882	0.118

Training auc is 1.0 in every row, so none of these controls stops the model from memorizing the training folds, they just change how much it hurts. On MIMIC row subsampling at 0.5 helps a bit (0.748 against 0.735 plain) and combining it with a minimum leaf of 5 gives the best result at 0.770, while the minimum leaf alone does nothing reliable (0.724 at 5, 0.741 at 10). That's one run of a noisy estimate with an sd of about 0.1 per fold, so I'm not claiming the 0.77 is real. On ECAMS the effects are tiny, 0.884 plain against 0.888 with a minimum leaf of 5 or 10. The one clear effect is that using every feature at each split (0.867) is worse than sampling sqrt of them (0.884), so feature subsampling works as regularization when the columns are this redundant.

In [6]:
```
# early stopping, let the model quit when a held out slice of the training fold stops improving
rows = []
for name in data:
    for nic in [5, 10, 20]:
        X, y = data[name]
        used = []
        aucs = []
        for a, b in cv_clf.split(X, y):
            m = gb_model(n_estimators = 500, learning_rate = 0.1, max_depth = 2, n_iter_no_change = nic, validation
            used.append(m[1].n_estimators_)
            aucs.append(roc_auc_score(y.iloc[b], m.predict_proba(X.iloc[b])[:, 1]))
        rows.append({'data': name, 'patience': nic, 'mean trees used': np.mean(used), 'cv auc': np.mean(aucs)})
pd.DataFrame(rows).round(3)
```

Out[6]:

	data	patience	mean trees used	cv auc
0	mimic	5	25.267	0.738
1	mimic	10	39.267	0.733
2	mimic	20	54.533	0.743
3	ecams	5	39.933	0.877
4	ecams	10	54.333	0.878
5	ecams	20	74.400	0.879

Early stopping holds the number of trees to what the data supports. With a patience of 5 to 20 rounds the model stops after about 25 to 55 trees on MIMIC and 40 to 74 trees on ECAMS, and the cv auc is about the same as the best fixed setting, 0.733 to 0.743 on MIMIC and 0.877 to 0.879 on ECAMS. It costs a bit of data since 20 percent of each training fold is held back, but it picks the number of rounds without me looking at the validation curve.

In [7]:
```
# nested tuning, a randomized search over the grid inside each outer fold, compared with the earlier weeks
grid = {'gradientboostingclassifier__learning_rate': [0.01, 0.03, 0.1, 0.2], 'gradientboostingclassifier__n_estimat
models = {
    'gradient boosting, default': make_pipeline(SimpleImputer(strategy = 'median'), GradientBoostingClassifier(rand
    'gradient boosting, tuned inside folds': RandomizedSearchCV(make_pipeline(SimpleImputer(strategy = 'median'), G
    'random forest, default': make_pipeline(SimpleImputer(strategy = 'median'), RandomForestClassifier(n_estimators
    'logistic, cv regularized': make_pipeline(SimpleImputer(strategy = 'median'), StandardScaler(), LogisticRegress
}
scoring = ('roc_auc', 'recall', 'precision', 'f1', 'accuracy')
```

```
rows = []
for name in data:
    X, y = data[name]
    for mn, m in models.items():
        r = cross_validate(m, X, y, cv = cv_clf, scoring = scoring, return_train_score = True, n_jobs = 2)
        rows.append({'data': name, 'model': mn, 'train auc': r['train_roc_auc'].mean(), 'auc': r['test_roc_auc'].me
        print(name, mn, round(rows[-1]['auc'], 3))
gb_tab = pd.DataFrame(rows)
gb_tab.to_csv('figs/w9_summary.csv', index = False)
gb_tab.round(3)
```

```
mimic gradient boosting, default 0.738
mimic gradient boosting, tuned inside folds 0.759
mimic random forest, default 0.779
mimic logistic, cv regularized 0.75
ecams gradient boosting, default 0.888
ecams gradient boosting, tuned inside folds 0.889
ecams random forest, default 0.898
ecams logistic, cv regularized 0.876
```

Out[7]:

	data	model	train auc	auc	auc sd	recall	precision	f1	accuracy
0	mimic	gradient boosting, default	1.000	0.738	0.105	0.541	0.599	0.558	0.733
1	mimic	gradient boosting, tuned inside folds	0.928	0.759	0.098	0.148	0.321	0.186	0.681
2	mimic	random forest, default	1.000	0.779	0.105	0.519	0.653	0.565	0.745
3	mimic	logistic, cv regularized	0.824	0.750	0.088	0.311	0.529	0.365	0.707
4	ecams	gradient boosting, default	1.000	0.888	0.024	0.869	0.851	0.859	0.806
5	ecams	gradient boosting, tuned inside folds	0.992	0.889	0.021	0.882	0.839	0.859	0.802
6	ecams	random forest, default	1.000	0.898	0.024	0.882	0.858	0.869	0.819
7	ecams	logistic, cv regularized	0.940	0.876	0.039	0.889	0.847	0.867	0.814

Tuning helps on MIMIC and does nothing on ECAMS. On MIMIC default boosting scores a cv auc of 0.738 and the tuned version 0.759, with the training auc dropping from 1.000 to 0.928, so the search found a less overfit model. The random forest is still ahead at 0.779. On ECAMS the default and tuned boosting are 0.888 and 0.889, below the forest (0.898) and above cv regularized logistic regression (0.876). The tuned MIMIC model has a recall of 0.15, the same problem as the tuned KNN last week, a small fast model with 32 percent positives rarely crosses 0.5. Boosting doesn't beat a random forest on either dataset, which I didn't expect, I thought the sequential correction would win.

In [8]:
```
# holdout check with a bootstrap interval, plus what the search picked on the full training rows
rng = np.random.default_rng(0)
rows = []
fitted = {}
for name in data:
    Xtr, Xte, ytr, yte = splits[name]
    for mn in ['gradient boosting, default', 'gradient boosting, tuned inside folds', 'random forest, default', 'lo
        m = models[mn].fit(Xtr, ytr)
        fitted[(name, mn)] = m
        p = m.predict_proba(Xte)[:, 1]
        yt = yte.values
        boots = []
        for b in range(1000):
            idx = rng.integers(0, len(yt), len(yt))
            if yt[idx].min() != yt[idx].max():
                boots.append(roc_auc_score(yt[idx], p[idx]))
        rows.append({'data': name, 'model': mn, 'test rows': len(yt), 'positives': int(yt.sum()), 'test auc': roc_a
    print(name, 'search picked', fitted[(name, 'gradient boosting, tuned inside folds')].best_params_)
pd.DataFrame(rows).round(3)
```

```
mimic search picked {'gradientboostingclassifier__subsample': 0.75, 'gradientboostingclassifier__n_estimators': 50,
'gradientboostingclassifier__min_samples_leaf': 10, 'gradientboostingclassifier__max_depth': 2, 'gradientboostingcla
ssifier__learning_rate': 0.2}
ecams search picked {'gradientboostingclassifier__subsample': 1.0, 'gradientboostingclassifier__n_estimators': 50, '
gradientboostingclassifier__min_samples_leaf': 5, 'gradientboostingclassifier__max_depth': 3, 'gradientboostingclass
ifier__learning_rate': 0.01}
```

Out[8]:

	data	model	test rows	positives	test auc	low	high
0	mimic	gradient boosting, default	28	9	0.842	0.598	1.000
1	mimic	gradient boosting, tuned inside folds	28	9	0.918	0.806	1.000
2	mimic	random forest, default	28	9	0.854	0.678	0.994
3	mimic	logistic, cv regularized	28	9	0.801	0.625	0.941
4	ecams	gradient boosting, default	75	51	0.874	0.790	0.945
5	ecams	gradient boosting, tuned inside folds	75	51	0.886	0.802	0.951
6	ecams	random forest, default	75	51	0.873	0.790	0.944
7	ecams	logistic, cv regularized	75	51	0.892	0.811	0.955

The holdout gives a tuned boosting auc of 0.918 on MIMIC (interval 0.81 to 1.00) and 0.886 on ECAMS (0.80 to 0.95). The MIMIC number is much higher than the cv estimate of 0.759, and with 28 test stays and 9 positives I take that as luck with the split. The same split gave 0.842 for default boosting and 0.854 for the forest. The search picked learning rate 0.2, 50 trees, depth 2, subsample 0.75 and a minimum leaf of 10 on MIMIC, a short, shallow, regularized model. On ECAMS it picked learning rate 0.01 with only 50 trees, which is barely trained, and still got about the same cv auc as the default, which says that on ECAMS almost any reasonable setting lands near 0.89.

In [9]:
```
# class weights through sample weights, auc against recall at the 0.5 cutoff
from sklearn.utils.class_weight import compute_sample_weight
rows = []
for name in data:
    X, y = data[name]
    for weighted in [False, True]:
        aucs, recs, precs = [], [], []
        for a, b in cv_clf.split(X, y):
            sw = compute_sample_weight('balanced', y.iloc[a]) if weighted else None
            m = gb_model(n_estimators = 100, learning_rate = 0.1, max_depth = 2)
            m.fit(X.iloc[a], y.iloc[a], gradientboostingclassifier__sample_weight = sw)
            pr = m.predict_proba(X.iloc[b])[:, 1]
            aucs.append(roc_auc_score(y.iloc[b], pr))
            recs.append(recall_score(y.iloc[b], pr > 0.5))
            precs.append(precision_score(y.iloc[b], pr > 0.5, zero_division = 0))
        rows.append({'data': name, 'balanced weights': weighted, 'auc': np.mean(aucs), 'recall': np.mean(recs), 'pr
pd.DataFrame(rows).round(3)
```

Out[9]:

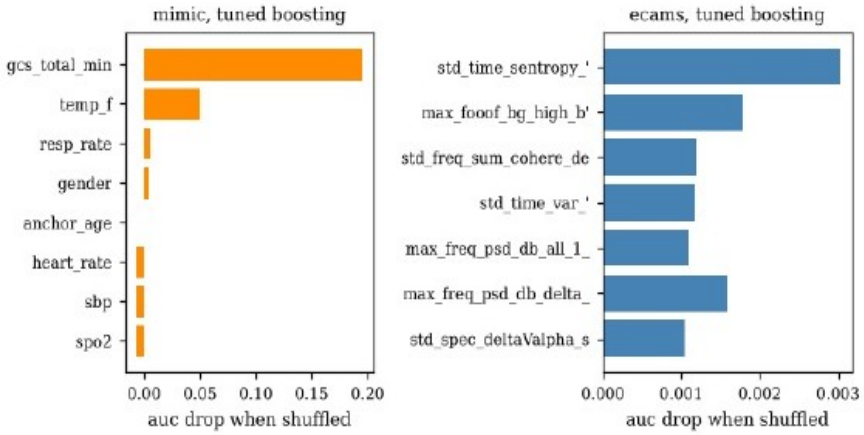
	data	balanced weights	auc	recall	precision
0	mimic	False	0.752	0.533	0.567
1	mimic	True	0.748	0.637	0.547
2	ecams	False	0.885	0.873	0.855
3	ecams	True	0.885	0.844	0.892

Balanced weights barely move the auc (0.752 against 0.748 on MIMIC, 0.885 for both on ECAMS) but they move the recall and precision. On MIMIC recall goes from 0.53 to 0.64 and precision slips from 0.57 to 0.55. On ECAMS it goes the other way, recall drops from 0.87 to 0.84 and precision climbs from 0.86 to 0.89, which makes sense since delirium is the majority class there and weighting pushes the model toward predicting the minority.

In [10]:
```
# figure, permutation importance for the tuned models, computed on held out folds
fig, ax = plt.subplots(1, 2, figsize = (6.5, 3.1))
for i, name in enumerate(data):
    X, y = data[name]
    best = fitted[(name, 'gradient boosting, tuned inside folds')].best_params_
    imps = []
    for a, b in StratifiedKFold(5, shuffle = True, random_state = 3).split(X, y):
        m = make_pipeline(SimpleImputer(strategy = 'median'), GradientBoostingClassifier(random_state = 0, max_feat
        p = permutation_importance(m, X.iloc[b], y.iloc[b], scoring = 'roc_auc', n_repeats = 5, random_state = 0)
        imps.append(p.importances_mean)
    imp = pd.Series(np.mean(imps, axis = 0), index = X.columns).sort_values(ascending = False).head(8)
    ax[i].barh([n[:22] for n in imp.index[::-1]], imp.values[::-1], color = 'darkorange' if name == 'mimic' else 's
    ax[i].set_xlabel('auc drop when shuffled')
    ax[i].set_title(name + ', tuned boosting')
    print(name, imp.round(3).to_dict())
plt.tight_layout()
plt.savefig('figs/w9_importance.png', dpi = 220)
```

```
plt.show()
```

```
mimic {'gcs_total_min': 0.196, 'temp_f': 0.05, 'resp_rate': 0.005, 'gender': 0.004, 'anchor_age': 0.0, 'heart_rate':
-0.006, 'sbp': -0.006, 'spo2': -0.007}
ecams {"std_time_sentropy_'": 0.003, "max_fooof_bg_high_b'": 0.002, "max_freq_psd_db_delta_2_6'": 0.002, "std_freq_s
um_cohere_delta_1_5'": 0.001, "std_time_var_'": 0.001, "max_freq_psd_db_all_1_20'": 0.001, "max_freq_psd_db_delta_1_
6'": 0.001, "std_spec_deltaValpha_std'": 0.001}
```



Importance is very concentrated on MIMIC and almost flat on ECAMS. On MIMIC shuffling gcs_total_min drops the auc by 0.20 and temp_f by 0.05, and everything else is under 0.01 or slightly negative, so most of the vitals aren't contributing. On ECAMS the top feature (std_time_sentropy) only drops the auc by 0.003 and the rest are smaller, because the model can get the same information from many near copies and shuffling one barely matters. That is the same result as the forest importance in Milestone One. The EEG ranking tells me which feature groups are used, mostly variability of delta band power and coherence, but not that any single feature is the driver.

To sum up, boosting reached about 0.76 on MIMIC and 0.89 on ECAMS, about the same as the other tuned models. The main overfitting lesson is that training auc goes to 1.0 almost immediately while validation peaks in the first 30 to 100 rounds on MIMIC, so rounds, learning rate and depth are really one knob that sets how fast the model memorizes.

Week 10 notebook, k means, elbow method and silhouette score, Mike Hembree

Same project as the earlier weeks. The capstone is an early warning model for ICU delirium on MIMIC IV, and the MIMIC part of this notebook runs on the MIMIC IV Clinical Database Demo (100 patients, 140 ICU stays) because the full credentialed access is still pending. The second dataset is the ECAMS EEG data from Massachusetts General Hospital (373 patients, 1189 EEG features, delirium called by the CAM algorithm), the same two datasets as the Milestone One notebooks. K means never sees the delirium label, so this week is about what structure is in the features. I use delirium afterwards just to describe the clusters. I ran k from 2 to 10 on MIMIC scaled and unscaled, on ECAMS scaled and unscaled, and on ECAMS after reducing to 10 principal components, since distances in 1189 redundant dimensions are not very informative. Every fit uses 20 random starts.

In [1]:
```
# imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, normalize
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, silhouette_samples, adjusted_rand_score

warnings.filterwarnings('ignore')
plt.rcParams['figure.dpi'] = 110
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 9
plt.rcParams['axes.titlesize'] = 9
plt.rcParams['axes.labelsize'] = 9
plt.rcParams['xtick.labelsize'] = 8
plt.rcParams['ytick.labelsize'] = 8
plt.rcParams['legend.fontsize'] = 7.5
```

In [2]:
```
# load both datasets, clustering never sees the delirium label, it is only used afterwards to describe the clusters
feat = pd.read_csv('delirium_features.csv')
mcols = ['anchor_age', 'gender', 'heart_rate', 'sbp', 'dbp', 'resp_rate', 'temp_f', 'spo2', 'gcs_total_min']
Xm = feat[mcols]
ym = feat['delirium_positive']

Xe = pd.read_csv('ecams/features_alldata.csv')
clin = pd.read_csv('ecams/E_CAM_S_Data.csv').sort_values('SubjectID-used_for_feature_extraction').reset_index(drop
ye = clin['CAM algorithm (CAM 1 & 2 , and 3 or 4 SATISFIED - DELIRIUM POSITIVE)'].astype(int)

Xm_filled = SimpleImputer(strategy = 'median').fit_transform(Xm)
spaces = {
    'mimic scaled': StandardScaler().fit_transform(Xm_filled),
    'mimic unscaled': Xm_filled,
    'ecams scaled': StandardScaler().fit_transform(Xe),
    'ecams unscaled': Xe.values,
    'ecams pca 10': PCA(10, random_state = 0).fit_transform(StandardScaler().fit_transform(Xe))
}
labels = {'mimic': ym.values, 'ecams': ye.values}
for k, v in spaces.items():
    print(k, v.shape)
```

```
mimic scaled (140, 9)
mimic unscaled (140, 9)
ecams scaled (373, 1189)
ecams unscaled (373, 1189)
ecams pca 10 (373, 10)
```

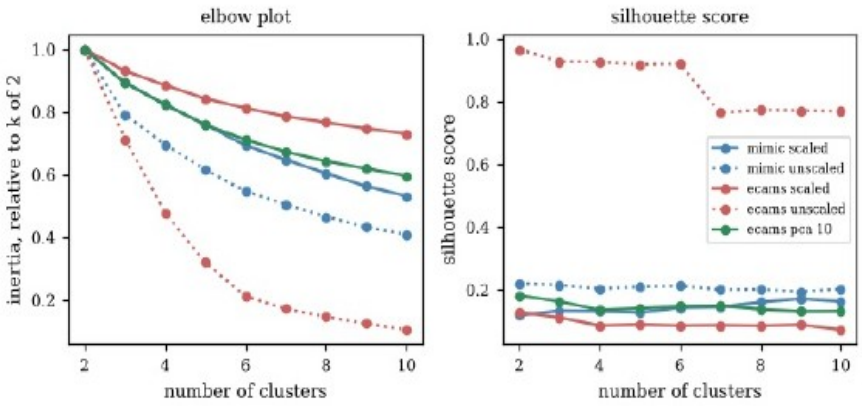
In [3]:
```
# elbow and silhouette for k from 2 to 10 in every feature space, 20 random starts each
rows = []
for sp, Z in spaces.items():
    for k in range(2, 11):
        km = KMeans(n_clusters = k, n_init = 20, random_state = 0).fit(Z)
        rows.append({'space': sp, 'k': k, 'inertia': km.inertia_, 'silhouette': silhouette_score(Z, km.labels_)})
ks_tab = pd.DataFrame(rows)
ks_tab.to_csv('figs/w10_k_sweep.csv', index = False)
ks_tab.pivot(index = 'k', columns = 'space', values = 'silhouette').round(3)
```

Out[3]:

space	ecams pca 10	ecams scaled	ecams unscaled	mimic scaled	mimic unscaled
k					
2	0.181	0.128	0.966	0.118	0.220
3	0.162	0.111	0.927	0.133	0.215
4	0.135	0.085	0.926	0.131	0.203
5	0.140	0.088	0.917	0.126	0.210
6	0.147	0.085	0.920	0.143	0.212
7	0.149	0.086	0.763	0.144	0.200
8	0.137	0.084	0.773	0.160	0.201
9	0.130	0.088	0.771	0.171	0.193
10	0.131	0.071	0.769	0.162	0.203

The silhouette scores are low almost everywhere. Scaled MIMIC sits between 0.12 and 0.17, scaled ECAMS between 0.07 and 0.13 and the 10 component ECAMS space between 0.13 and 0.18. A silhouette under about 0.25 means the clusters aren't well separated, and I don't think either dataset has real cluster structure in the feature space. The two exceptions are the unscaled runs, MIMIC unscaled at about 0.20 to 0.22, and ECAMS unscaled at 0.97, which I look at below.

In [4]:
```
# figure, elbow on the left and silhouette on the right
fig, ax = plt.subplots(1, 2, figsize = (6.5, 2.9))
sty = {'mimic scaled': ('steelblue', '-'), 'mimic unscaled': ('steelblue', ':'), 'ecams scaled': ('indianred', '-')
for sp, (c, ls) in sty.items():
    t = ks_tab[ks_tab.space == sp]
    ax[0].plot(t['k'], t['inertia'] / t['inertia'].iloc[0], 'o', color = c, linestyle = ls, label = sp, markersize
    ax[1].plot(t['k'], t['silhouette'], 'o', color = c, linestyle = ls, label = sp, markersize = 4)
ax[0].set_xlabel('number of clusters')
ax[0].set_ylabel('inertia, relative to k of 2')
ax[0].set_title('elbow plot')
ax[1].set_xlabel('number of clusters')
ax[1].set_ylabel('silhouette score')
ax[1].set_title('silhouette score')
ax[1].legend(fontsize = 6.5)
plt.tight_layout()
plt.savefig('figs/w10_elbow_sil.png', dpi = 220)
plt.show()
```



The elbow plot isn't much use. Inertia falls smoothly in every space with no bend, so the elbow gives no answer for k. The silhouette gives answers, but they are weak ones. Scaled MIMIC is highest at k of 9 (0.171) with a flat stretch from 6 to 10 and ECAMS is best at k of 2 in both the scaled (0.128) and PCA (0.181) versions.

In [5]:
```
# what scaling does, the feature that dominates the distance in the unscaled spaces
for nm, X in [('mimic', Xm), ('ecams', Xe)]:
    v = X.var()
    print(nm, 'share of total variance from the biggest feature:', round(v.max() / v.sum(), 3), v.idxmax())
    print(nm, 'features that make up 95 percent of the variance:', int((v.sort_values(ascending = False).cumsum() /
```

```
mimic share of total variance from the biggest feature: 0.31 sbp
mimic features that make up 95 percent of the variance: 5 of 9
ecams share of total variance from the biggest feature: 0.213 max_ratio_psd_db_deltathetaValphabetamid'
ecams features that make up 95 percent of the variance: 18 of 1189
```

This is why the unscaled numbers look so different. On MIMIC systolic pressure alone is 31 percent of the total variance and 5 of 9 features make up 95 percent. On ECAMS it's far more extreme, 18 of the 1189 features carry 95 percent of the variance.

In [6]:
```
# the unscaled spaces, cluster sizes at k = 2 and what the silhouette is really rewarding
for sp, nm in [('mimic unscaled', 'mimic'), ('ecams unscaled', 'ecams')]:
    km = KMeans(n_clusters = 2, n_init = 20, random_state = 0).fit(spaces[sp])
    sizes = np.bincount(km.labels_)
    print(sp, 'cluster sizes', sizes.tolist(), 'silhouette', round(silhouette_score(spaces[sp], km.labels_), 3), 'a
Zu = Xe.values
d = pd.Series(np.abs(Zu - Zu.mean(axis = 0)).mean(axis = 0), index = Xe.columns).sort_values(ascending = False)
print('ecams unscaled, the 3 features with the largest average spread:', d.head(3).round(0).to_dict())
```

```
mimic unscaled cluster sizes [50, 90] silhouette 0.22 ari with delirium -0.016
ecams unscaled cluster sizes [372, 1] silhouette 0.966 ari with delirium 0.006
ecams unscaled, the 3 features with the largest average spread: {"min_ratio_psd_db_deltathetamidVtheta'": 3392.0, "m
ax_ratio_psd_db_deltathetaValphabetamid'": 3258.0, "max_ratio_psd_db_deltathetaValphalow'": 3243.0}
```

The 0.966 silhouette for unscaled ECAMS isn't a good clustering, it's one cluster of 372 patients and one patient on their own. The scales are so uneven that a single EEG ratio feature (the largest average spread is about 3400) pulls one outlier away from everyone. The silhouette is rewarding that. Unscaled MIMIC splits 50 and 90 with a silhouette of 0.22 and has an ARI with delirium of negative 0.016, so that structure has nothing to do with delirium. This is the main thing I'd warn about when using silhouette, a high score can come from a degenerate or scale driven solution.

In [7]:
```
# silhouette under different distance metrics for the same k means labels, and seed stability
rows = []
for sp in ['mimic scaled', 'ecams scaled', 'ecams pca 10']:
    Z = spaces[sp]
    for k in [2, 3, 4, 5]:
        km = KMeans(n_clusters = k, n_init = 20, random_state = 0).fit(Z)
        row = {'space': sp, 'k': k}
        for met in ['euclidean', 'manhattan', 'cosine']:
            row[met] = silhouette_score(Z, km.labels_, metric = met)
        runs = [KMeans(n_clusters = k, n_init = 1, random_state = s).fit(Z).labels_ for s in range(10)]
        aris = [adjusted_rand_score(runs[i], runs[j]) for i in range(10) for j in range(i + 1, 10)]
        row['seed ari mean'] = np.mean(aris)
        row['seed ari min'] = np.min(aris)
        rows.append(row)
metric_tab = pd.DataFrame(rows)
metric_tab.to_csv('figs/w10_metrics_seeds.csv', index = False)
metric_tab.round(3)
```

Out[7]:

	space	k	euclidean	manhattan	cosine	seed ari mean	seed ari min
0	mimic scaled	2	0.118	0.124	0.216	0.176	-0.005
1	mimic scaled	3	0.133	0.147	0.245	0.363	0.027
2	mimic scaled	4	0.131	0.132	0.250	0.299	0.158
3	mimic scaled	5	0.126	0.137	0.133	0.375	0.187
4	ecams scaled	2	0.128	0.155	0.238	0.935	0.875
5	ecams scaled	3	0.111	0.132	0.202	0.701	0.420
6	ecams scaled	4	0.085	0.108	0.164	0.471	0.322
7	ecams scaled	5	0.088	0.108	0.166	0.472	0.320
8	ecams pca 10	2	0.181	0.151	0.322	0.941	0.875
9	ecams pca 10	3	0.162	0.130	0.281	0.670	0.351
10	ecams pca 10	4	0.135	0.116	0.247	0.476	0.273
11	ecams pca 10	5	0.140	0.124	0.236	0.564	0.264

Distance metric changes the silhouette more than I expected. For the same K means labels, cosine silhouette is nearly double the euclidean one (scaled MIMIC at k of 3 is 0.245 against 0.133, scaled ECAMS at k of 2 is 0.238 against 0.128, the PCA space is 0.322 against 0.181). K means itself minimizes euclidean distance, so the cosine version isn't a fair measure of what the algorithm optimized, but it does say the clusters are more separated by direction than by distance. The seed check shows how unstable some of these are. On ECAMS k of 2 the ten single start runs agree almost perfectly (mean ARI 0.935, worst pair 0.875) and at k of 4 they

fall to 0.47. On MIMIC even k of 2 only agrees at 0.18 on average (worst pair about 0) which means a single start would give a different answer each time.

In [8]:
```
# spherical k means, normalize every row to length one first so the clusters follow direction and not size
rows = []
for sp in ['mimic scaled', 'ecams scaled']:
    Z = normalize(spaces[sp])
    for k in [2, 3, 4, 5, 6]:
        km = KMeans(n_clusters = k, n_init = 20, random_state = 0).fit(Z)
        rows.append({'space': sp + ', row normalized', 'k': k, 'silhouette cosine': silhouette_score(Z, km.labels_,
pd.DataFrame(rows).round(3)
```

Out[8]:

	space	k	silhouette cosine	ari with delirium
0	mimic scaled, row normalized	2	0.241	0.118
1	mimic scaled, row normalized	3	0.255	0.028
2	mimic scaled, row normalized	4	0.266	0.013
3	mimic scaled, row normalized	5	0.257	0.015
4	mimic scaled, row normalized	6	0.286	0.011
5	ecams scaled, row normalized	2	0.238	0.218
6	ecams scaled, row normalized	3	0.203	0.107
7	ecams scaled, row normalized	4	0.187	0.115
8	ecams scaled, row normalized	5	0.183	0.066
9	ecams scaled, row normalized	6	0.166	0.068

Normalizing the rows to length one first (so K means follows direction, like cosine distance) gives silhouettes of 0.24 to 0.29 on MIMIC and 0.17 to 0.24 on ECAMS, a little better than plain euclidean but still low. The ARI with delirium stays small on MIMIC (0.01 to 0.12) and reaches 0.218 on ECAMS at k of 2.

In [9]:
```
# cluster profiles at the silhouette best k and at k = 4, delirium rate and size in every cluster
rows = []
for sp, nm in [('mimic scaled', 'mimic'), ('ecams scaled', 'ecams'), ('ecams pca 10', 'ecams')]:
    sub = ks_tab[ks_tab.space == sp]
    kbest = int(sub.loc[sub['silhouette'].idxmax(), 'k'])
    for k in sorted({kbest, 4}):
        km = KMeans(n_clusters = k, n_init = 20, random_state = 0).fit(spaces[sp])
        for c in range(k):
            mk = km.labels_ == c
            rows.append({'space': sp, 'k': k, 'cluster': c, 'size': int(mk.sum()), 'delirium rate': labels[nm][mk].
prof = pd.DataFrame(rows)
prof.to_csv('figs/w10_profiles.csv', index = False)
prof.round(3)
```

Out[9]:

	space	k	cluster	size	delirium rate	ari with delirium
0	mimic scaled	4	0	39	0.256	-0.003
1	mimic scaled	4	1	28	0.321	-0.003
2	mimic scaled	4	2	40	0.400	-0.003
3	mimic scaled	4	3	33	0.303	-0.003
4	mimic scaled	9	0	22	0.364	0.020
5	mimic scaled	9	1	16	0.062	0.020
6	mimic scaled	9	2	20	0.200	0.020
7	mimic scaled	9	3	25	0.320	0.020
8	mimic scaled	9	4	14	0.214	0.020
9	mimic scaled	9	5	1	0.000	0.020
10	mimic scaled	9	6	11	0.091	0.020
11	mimic scaled	9	7	14	0.643	0.020
12	mimic scaled	9	8	17	0.647	0.020
13	ecams scaled	2	0	184	0.924	0.223
14	ecams scaled	2	1	189	0.444	0.223
15	ecams scaled	4	0	67	0.925	0.089
16	ecams scaled	4	1	93	0.333	0.089
17	ecams scaled	4	2	126	0.611	0.089
18	ecams scaled	4	3	87	0.966	0.089
19	ecams pca 10	2	0	189	0.444	0.223
20	ecams pca 10	2	1	184	0.924	0.223
21	ecams pca 10	4	0	115	0.652	0.103
22	ecams pca 10	4	1	87	0.379	0.103
23	ecams pca 10	4	2	111	0.964	0.103
24	ecams pca 10	4	3	60	0.650	0.103

On ECAMS the two cluster solution splits the patients into 184 with a delirium rate of 92.4 percent and 189 with 44.4 percent. The adjusted Rand index against delirium is 0.22, which is low in absolute terms, but a gap between 92 and 44 percent from clustering that never saw the label is a real finding. The 10 component version gives the same split. At k of 4 the rates range from 33 to 97 percent. On MIMIC none of it lines up with delirium, at k of 4 the rates are 26, 32, 40 and 30 percent and the ARI is about zero. The k of 9 solution has rates from 6 to 65 percent but the clusters have 11 to 25 stays each so those differences could easily be noise.

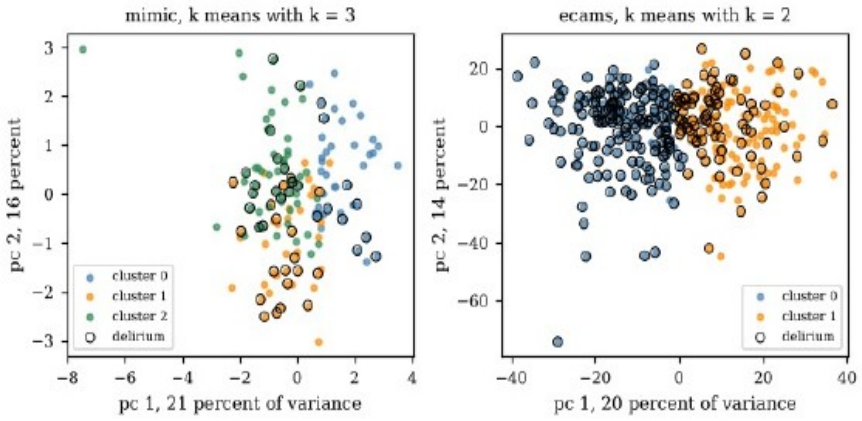
In [10]:
```
# what the mimic clusters look like in the original units
km = KMeans(n_clusters = 3, n_init = 20, random_state = 0).fit(spaces['mimic scaled'])
tab = pd.DataFrame(Xm_filled, columns = mcols)
tab['cluster'] = km.labels_
tab['delirium'] = ym.values
tab.groupby('cluster').agg(['mean']).round(2).T
```

Out[10]:

	cluster	0	1	2
anchor_age	mean	60.24	58.05	66.51
gender	mean	0.49	0.86	0.35
heart_rate	mean	81.44	90.17	92.84
sbp	mean	132.23	115.47	108.72
dbp	mean	72.93	61.92	59.37
resp_rate	mean	17.46	19.47	21.26
temp_f	mean	98.32	98.72	98.26
spo2	mean	96.70	97.16	96.25
gcs_total_min	mean	12.71	3.68	11.98
delirium	mean	0.24	0.43	0.29

The three MIMIC clusters (41, 44 and 55 stays) are split mostly by GCS and blood pressure. Cluster 1 has an average minimum GCS of 3.7 against about 12 to 13 in the others, it's 86 percent male and has the highest delirium rate at 43 percent. Cluster 0 has the highest blood pressure (systolic 132) and the lowest delirium rate at 24 percent, cluster 2 is the oldest group (66.5) with the lowest blood pressure and a rate of 29 percent. So the clusters do pick up clinically sensible groups, comatose patients, hypertensive patients and older hypotensive patients, but the delirium rates only differ by 24 to 43 percent on groups of 40 to 55 stays.

In [11]:
```
# figure, clusters in the first two principal components, colored by cluster and outlined by delirium
fig, ax = plt.subplots(1, 2, figsize = (6.5, 3.0))
for i, (sp, nm, k) in enumerate([('mimic scaled', 'mimic', 3), ('ecams scaled', 'ecams', 2)]):
    Z = spaces[sp]
    km = KMeans(n_clusters = k, n_init = 20, random_state = 0).fit(Z)
    pc = PCA(2, random_state = 0).fit(Z)
    P = pc.transform(Z)
    pal = ['steelblue', 'darkorange', 'seagreen', 'purple']
    for c in range(k):
        mk = km.labels_ == c
        ax[i].scatter(P[mk, 0], P[mk, 1], s = 9, color = pal[c], alpha = 0.7, label = 'cluster ' + str(c))
    pos = labels[nm] == 1
    ax[i].scatter(P[pos, 0], P[pos, 1], s = 22, facecolors = 'none', edgecolors = 'black', linewidths = 0.7, label
    ax[i].set_xlabel('pc 1, ' + str(round(pc.explained_variance_ratio_[0] * 100)) + ' percent of variance')
    ax[i].set_ylabel('pc 2, ' + str(round(pc.explained_variance_ratio_[1] * 100)) + ' percent')
    ax[i].set_title(nm + ', k means with k = ' + str(k))
    ax[i].legend(fontsize = 6.5)
plt.tight_layout()
plt.savefig('figs/w10_pca_clusters.png', dpi = 220)
plt.show()
```



The principal component plots show why the silhouettes are low. The MIMIC clusters overlap in the middle and the delirium points are scattered through all three. ECAMS has a clear left to right gradient and the two clusters are more like two halves of one cloud than two separate groups, although the delirium points are noticeably denser in the orange half.

To sum up, silhouette said k of 2 for ECAMS and nothing clear for MIMIC, scaling is required, and the ECAMS split lines up with delirium better than I expected for an unsupervised method.

Week 11 notebook, DBSCAN and hierarchical clustering, Mike Hembree

Same project as the earlier weeks. The capstone is an early warning model for ICU delirium on MIMIC IV, and the MIMIC part of this notebook runs on the MIMIC IV Clinical Database Demo (100 patients, 140 ICU stays) because the full credentialed access is still pending. The second dataset is the ECAMS EEG data from Massachusetts General Hospital (373 patients, 1189 EEG features, delirium called by the CAM algorithm), the same two datasets as the Milestone One notebooks. Again the clustering never sees the label. DBSCAN needs an eps and a min_samples and marks points as noise if they aren't dense enough, so I used the k distance plots to pick the eps range for each space. For hierarchical clustering I compared ward, complete, average and single linkage and looked at the dendrograms, the cophenetic correlation, and how lopsided the cut is.

In [1]:
```
# imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
from scipy.cluster.hierarchy import linkage, dendrogram, cophenet, fcluster
from scipy.spatial.distance import pdist

warnings.filterwarnings('ignore')
plt.rcParams['figure.dpi'] = 110
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 9
plt.rcParams['axes.titlesize'] = 9
plt.rcParams['axes.labelsize'] = 9
plt.rcParams['xtick.labelsize'] = 8
plt.rcParams['ytick.labelsize'] = 8
plt.rcParams['legend.fontsize'] = 7.5
```

In [2]:
```
# load both datasets, clustering never sees the delirium label, it is only used afterwards to describe the clusters
feat = pd.read_csv('delirium_features.csv')
mcols = ['anchor_age', 'gender', 'heart_rate', 'sbp', 'dbp', 'resp_rate', 'temp_f', 'spo2', 'gcs_total_min']
Xm = feat[mcols]
ym = feat['delirium_positive']

Xe = pd.read_csv('ecams/features_alldata.csv')
clin = pd.read_csv('ecams/E_CAM_S_Data.csv').sort_values('SubjectID-used_for_feature_extraction').reset_index(drop
ye = clin['CAM algorithm (CAM 1 & 2 , and 3 or 4 SATISFIED - DELIRIUM POSITIVE)'].astype(int)

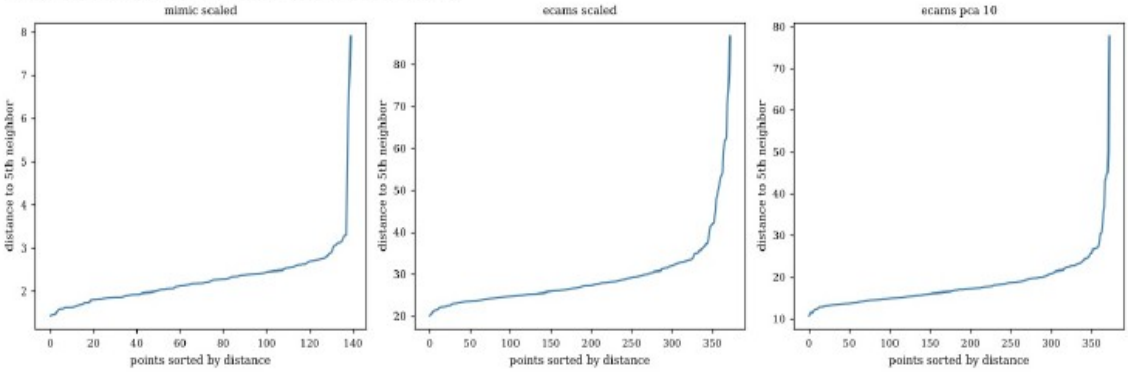
Xm_filled = SimpleImputer(strategy = 'median').fit_transform(Xm)
spaces = {
    'mimic scaled': StandardScaler().fit_transform(Xm_filled),
    'mimic unscaled': Xm_filled,
    'ecams scaled': StandardScaler().fit_transform(Xe),
    'ecams unscaled': Xe.values,
    'ecams pca 10': PCA(10, random_state = 0).fit_transform(StandardScaler().fit_transform(Xe))
}
labels = {'mimic': ym.values, 'ecams': ye.values}
for k, v in spaces.items():
    print(k, v.shape)
```

```
mimic scaled (140, 9)
mimic unscaled (140, 9)
ecams scaled (373, 1189)
ecams unscaled (373, 1189)
ecams pca 10 (373, 10)
```

In [3]:
```
# k distance curves, distance to the 5th nearest neighbor sorted, the knee suggests eps
fig, ax = plt.subplots(1, 3, figsize = (13, 4))
for i, sp in enumerate(['mimic scaled', 'ecams scaled', 'ecams pca 10']):
    Z = spaces[sp]
    d = np.sort(NearestNeighbors(n_neighbors = 5).fit(Z).kneighbors(Z)[0][:, -1])
    ax[i].plot(d, color = 'steelblue')
    ax[i].set_xlabel('points sorted by distance')
    ax[i].set_ylabel('distance to 5th neighbor')
    ax[i].set_title(sp)
    print(sp, 'median', round(np.median(d), 2), '90th percentile', round(np.percentile(d, 90), 2))
plt.tight_layout()
plt.savefig('figs/w11_kdist.png', dpi = 200)
```

```
plt.show()
```

```
mimic scaled median 2.18 90th percentile 2.74
ecams scaled median 26.71 90th percentile 35.6
ecams pca 10 median 16.84 90th percentile 23.17
```



None of the curves has a clean knee. They rise slowly and then shoot up for the last 5 to 10 percent of points. For scaled MIMIC the distance to the 5th neighbor has a median of 2.18 and a 90th percentile of 2.74, for scaled ECAMS it's 26.7 and 35.6, and for the 10 component ECAMS space 16.8 and 23.2. I used those as the center of the eps grids.

In [4]:
```
# dbscan grid, clusters found, share of noise, silhouette on the clustered points and delirium rate in the noise
rows = []
grids = {'mimic scaled': [1.5, 1.75, 2, 2.25, 2.5, 3], 'ecams scaled': [22, 25, 28, 31, 34, 38, 45], 'ecams pca 10'
for sp, eps_list in grids.items():
    nm = sp.split()[0]
    Z = spaces[sp]
    for eps in eps_list:
        for ms in [3, 5, 10]:
            lab = DBSCAN(eps = eps, min_samples = ms).fit_predict(Z)
            core = lab != -1
            ncl = len(set(lab[core]))
            row = {'space': sp, 'eps': eps, 'min samples': ms, 'clusters': ncl, 'noise share': 1 - core.mean()}
            row['silhouette'] = silhouette_score(Z[core], lab[core]) if ncl > 1 else np.nan
            row['delirium rate in noise'] = labels[nm][~core].mean() if (~core).sum() > 0 else np.nan
            row['delirium rate in clusters'] = labels[nm][core].mean() if core.sum() > 0 else np.nan
            rows.append(row)
db_tab = pd.DataFrame(rows)
db_tab.to_csv('figs/w11_dbscan_grid.csv', index = False)
db_tab.round(3)
```

Out[4]:

	space	eps	min samples	clusters	noise share	silhouette	delirium rate in noise	delirium rate in clusters
0	mimic scaled	1.50	3	5	0.786	0.426	0.309	0.367
1	mimic scaled	1.50	5	2	0.929	0.163	0.323	0.300
2	mimic scaled	1.50	10	0	1.000	NaN	0.321	NaN
3	mimic scaled	1.75	3	7	0.536	0.218	0.280	0.369
4	mimic scaled	1.75	5	3	0.757	0.361	0.292	0.412
5	mimic scaled	1.75	10	1	0.929	NaN	0.323	0.300
6	mimic scaled	2.00	3	4	0.286	0.109	0.325	0.320
7	mimic scaled	2.00	5	3	0.436	0.182	0.279	0.354
8	mimic scaled	2.00	10	3	0.579	0.255	0.247	0.424
9	mimic scaled	2.25	3	1	0.129	NaN	0.222	0.336
10	mimic scaled	2.25	5	1	0.164	NaN	0.348	0.316
11	mimic scaled	2.25	10	1	0.336	NaN	0.277	0.344
12	mimic scaled	2.50	3	1	0.071	NaN	0.300	0.323
13	mimic scaled	2.50	5	1	0.093	NaN	0.231	0.331
14	mimic scaled	2.50	10	1	0.121	NaN	0.235	0.333
15	mimic scaled	3.00	3	1	0.014	NaN	0.000	0.326
16	mimic scaled	3.00	5	1	0.014	NaN	0.000	0.326

17	mimic scaled	3.00	10	1	0.021	NaN	0.000	0.328
18	ecams scaled	22.00	3	6	0.791	0.143	0.647	0.808
19	ecams scaled	22.00	5	3	0.879	0.185	0.649	0.911
20	ecams scaled	22.00	10	0	1.000	NaN	0.681	NaN
21	ecams scaled	25.00	3	1	0.429	NaN	0.638	0.714
22	ecams scaled	25.00	5	1	0.464	NaN	0.636	0.720
23	ecams scaled	25.00	10	1	0.622	NaN	0.625	0.773
24	ecams scaled	28.00	3	1	0.257	NaN	0.677	0.682
25	ecams scaled	28.00	5	1	0.260	NaN	0.670	0.685
26	ecams scaled	28.00	10	1	0.282	NaN	0.648	0.694
27	ecams scaled	31.00	3	1	0.134	NaN	0.760	0.669
28	ecams scaled	31.00	5	1	0.134	NaN	0.760	0.669
29	ecams scaled	31.00	10	1	0.145	NaN	0.759	0.668
30	ecams scaled	34.00	3	1	0.088	NaN	0.788	0.671
31	ecams scaled	34.00	5	1	0.088	NaN	0.788	0.671
32	ecams scaled	34.00	10	1	0.088	NaN	0.788	0.671
33	ecams scaled	38.00	3	1	0.067	NaN	0.800	0.672
34	ecams scaled	38.00	5	1	0.067	NaN	0.800	0.672
35	ecams scaled	38.00	10	1	0.067	NaN	0.800	0.672
36	ecams scaled	45.00	3	1	0.048	NaN	0.833	0.673
37	ecams scaled	45.00	5	1	0.048	NaN	0.833	0.673
38	ecams scaled	45.00	10	1	0.048	NaN	0.833	0.673
39	ecams pca 10	13.00	3	12	0.662	0.051	0.684	0.675
40	ecams pca 10	13.00	5	5	0.847	0.237	0.665	0.772
41	ecams pca 10	13.00	10	2	0.949	0.145	0.681	0.684
42	ecams pca 10	15.00	3	6	0.386	-0.090	0.729	0.651
43	ecams pca 10	15.00	5	2	0.480	-0.019	0.715	0.649
44	ecams pca 10	15.00	10	4	0.716	0.194	0.652	0.755
45	ecams pca 10	17.00	3	3	0.225	0.045	0.690	0.678
46	ecams pca 10	17.00	5	1	0.279	NaN	0.702	0.673
47	ecams pca 10	17.00	10	1	0.362	NaN	0.704	0.668
48	ecams pca 10	19.00	3	3	0.110	0.048	0.707	0.678
49	ecams pca 10	19.00	5	1	0.134	NaN	0.740	0.672
50	ecams pca 10	19.00	10	1	0.220	NaN	0.732	0.667
51	ecams pca 10	21.00	3	1	0.080	NaN	0.667	0.682
52	ecams pca 10	21.00	5	1	0.086	NaN	0.688	0.680
53	ecams pca 10	21.00	10	1	0.107	NaN	0.700	0.679
54	ecams pca 10	24.00	3	1	0.046	NaN	0.765	0.677
55	ecams pca 10	24.00	5	1	0.051	NaN	0.737	0.678
56	ecams pca 10	24.00	10	1	0.054	NaN	0.700	0.680
57	ecams pca 10	28.00	3	1	0.021	NaN	1.000	0.674
58	ecams pca 10	28.00	5	1	0.024	NaN	1.000	0.673
59	ecams pca 10	28.00	10	1	0.024	NaN	1.000	0.673

DBSCAN doesn't find real clusters on either dataset. For every eps, either almost every point is in one big cluster with a few noise

points, or most of the points are noise. On MIMIC at eps of 2.0 and min_samples of 5 it finds 3 clusters but 44 percent of the stays are noise and the silhouette is 0.18, and at 2.25 and above it's one cluster. On scaled ECAMS the pattern is the same, eps of 22 gives 3 small clusters with 88 percent noise, and eps of 25 and up gives one cluster. The 10 component version is no better, eps of 15 gives two clusters with a silhouette of negative 0.02.

What the noise looks like is more interesting on ECAMS. At an eps of 34 about 9 percent of patients are noise and 79 percent of them are delirious against 67 percent of the clustered patients, and at an eps of 45 it's 83 percent of the 5 percent flagged as noise. These are patients with unusual EEG feature values, and they are somewhat more likely to be delirious. On MIMIC the noise is not enriched, 23 to 35 percent against 32 to 35 percent in the cluster.

In [5]:
```
# dbscan on the unscaled and scaled mimic data, what the scaling does to the density
rows = []
for sp in ['mimic unscaled', 'mimic scaled']:
    Z = spaces[sp]
    d = np.sort(NearestNeighbors(n_neighbors = 5).fit(Z).kneighbors(Z)[0][:, -1])
    eps = np.percentile(d, 85)
    lab = DBSCAN(eps = eps, min_samples = 5).fit_predict(Z)
    rows.append({'space': sp, 'eps at 85th percentile': eps, 'clusters': len(set(lab)) - (1 if -1 in lab else 0), '
pd.DataFrame(rows).round(3)
```

Out[5]:

	space	eps at 85th percentile	clusters	noise share
0	mimic unscaled	20.344	1	0.071
1	mimic scaled	2.629	1	0.057

Using the 85th percentile of the 5th neighbor distance as eps gives one cluster on both the unscaled and the scaled MIMIC data, with 7 and 6 percent noise.

In [6]:
```
# hierarchical clustering, four linkages on both datasets, cophenetic correlation and how lopsided the cut is
rows = []
links = {}
for sp in ['mimic scaled', 'ecams scaled']:
    nm = sp.split()[0]
    Z = spaces[sp]
    dist = pdist(Z)
    for meth in ['ward', 'complete', 'average', 'single']:
        L = linkage(Z, method = meth)
        links[(sp, meth)] = L
        c, _ = cophenet(L, dist)
        for k in [2, 3, 4, 5]:
            lab = fcluster(L, k, criterion = 'maxclust')
            sizes = np.bincount(lab)[1:]
            rows.append({'space': sp, 'linkage': meth, 'k': k, 'cophenetic': c, 'silhouette': silhouette_score(Z, l
hac_tab = pd.DataFrame(rows)
hac_tab.to_csv('figs/w11_hac.csv', index = False)
hac_tab[hac_tab.k.isin([3, 4])].round(3)
```

Out[6]:

	space	linkage	k	cophenetic	silhouette	largest cluster share	smallest cluster	ari with delirium
1	mimic scaled	ward	3	0.354	0.114	0.400	36	0.006
2	mimic scaled	ward	4	0.354	0.118	0.393	1	0.003
5	mimic scaled	complete	3	0.583	0.466	0.986	1	-0.015
6	mimic scaled	complete	4	0.583	0.066	0.721	1	0.072
9	mimic scaled	average	3	0.788	0.466	0.986	1	-0.015
10	mimic scaled	average	4	0.788	0.221	0.979	1	-0.022
13	mimic scaled	single	3	0.747	0.466	0.986	1	-0.015
14	mimic scaled	single	4	0.747	0.116	0.979	1	0.001
17	ecams scaled	ward	3	0.287	0.095	0.426	68	0.130
18	ecams scaled	ward	4	0.287	0.049	0.426	60	0.052
21	ecams scaled	complete	3	0.609	0.316	0.965	2	-0.025
22	ecams scaled	complete	4	0.609	0.315	0.957	2	-0.017
25	ecams scaled	average	3	0.824	0.482	0.995	1	-0.006
26	ecams scaled	average	4	0.824	0.461	0.992	1	0.000
29	ecams scaled	single	3	0.780	0.479	0.995	1	0.003
30	ecams scaled	single	4	0.780	0.426	0.992	1	0.009

Linkage choice decides everything. Complete, average and single linkage give a huge cluster and a few tiny ones. For three clusters on ECAMS the largest cluster holds 96.5 percent of the patients with complete linkage and 99.5 percent with average and single, and on MIMIC it's 98.6 percent for all three. Their silhouettes are high (0.32 to 0.48 on ECAMS, 0.47 on MIMIC) but only because almost everything is in one cluster and the other clusters are one or two points, so the number is meaningless. This is chaining. The cophenetic correlation is also higher for those linkages (0.82 for average on ECAMS, 0.79 on MIMIC) than for ward (0.29 and 0.35), because it measures how well the tree preserves the pairwise distances, not whether the clusters are useful. Ward is the only linkage that makes balanced clusters, the largest has 43 percent of ECAMS and 40 percent of MIMIC at three clusters, even though its silhouette is the lowest (0.095 on ECAMS and 0.114 on MIMIC). On MIMIC at four clusters ward splits off one stay on its own.

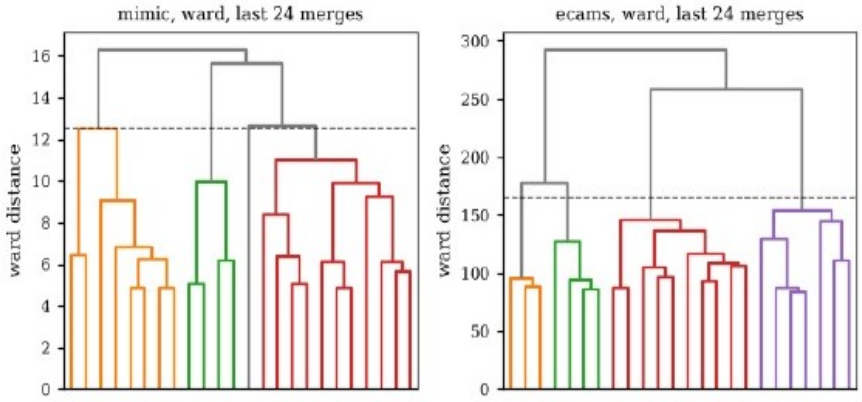
In [7]:
```
# ward clusters at k = 4, size and delirium rate in each
rows = []
for sp in ['mimic scaled', 'ecams scaled']:
    nm = sp.split()[0]
    lab = fcluster(links[(sp, 'ward')], 4, criterion = 'maxclust')
    for c in range(1, 5):
        mk = lab == c
        rows.append({'space': sp, 'cluster': c, 'size': int(mk.sum()), 'delirium rate': labels[nm][mk].mean()})
pd.DataFrame(rows).round(3)
```

Out[7]:

	space	cluster	size	delirium rate
0	mimic scaled	1	48	0.271
1	mimic scaled	2	36	0.417
2	mimic scaled	3	55	0.309
3	mimic scaled	4	1	0.000
4	ecams scaled	1	60	0.850
5	ecams scaled	2	86	0.977
6	ecams scaled	3	159	0.453
7	ecams scaled	4	68	0.691

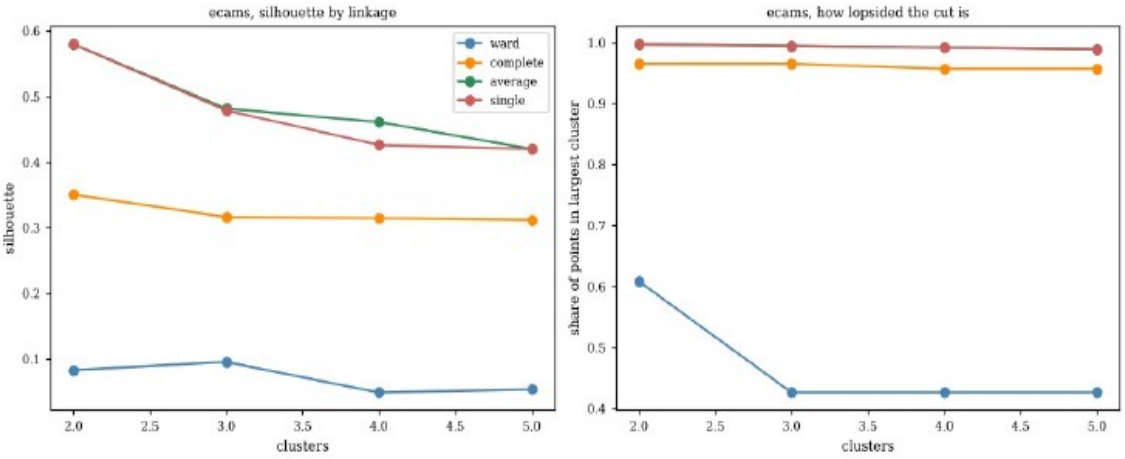
The ward clusters on ECAMS have delirium rates of 85 percent (60 patients), 98 percent (86), 45 percent (159) and 69 percent (68). That is a much wider spread than the overall rate of 68 percent and the cluster with 98 percent is the one that gets most of the delirious patients. The ARI is only 0.13 at three clusters because the groups are different sizes and the rates aren't 0 and 100. On MIMIC the rates are 27, 42 and 31 percent for the three real clusters (48, 36 and 55 stays), so ward finds the same story as K means, a bit of signal but nothing a clinician could use.

In [8]:
```
# figure, dendrograms for the ward linkage on both datasets, cut at four clusters
fig, ax = plt.subplots(1, 2, figsize = (6.5, 2.9))
for i, sp in enumerate(['mimic scaled', 'ecams scaled']):
    L = links[(sp, 'ward')]
    thr = (L[-4, 2] + L[-3, 2]) / 2
    dendrogram(L, truncate_mode = 'lastp', p = 24, ax = ax[i], color_threshold = thr, no_labels = True, above_thres
    ax[i].axhline(thr, color = 'black', linestyle = '--', linewidth = 0.8)
    ax[i].set_ylabel('ward distance')
    ax[i].set_title(sp.split()[0] + ', ward, last 24 merges')
plt.tight_layout()
plt.savefig('figs/w11_dendro.png', dpi = 220)
plt.show()
```



The ECAMS dendrogram has two big branches that split at a ward distance of about 290, and the four cluster cut is easy to see. The MIMIC tree is much more even, there isn't one height where the cut is obviously right, which matches its low silhouette and cophenetic correlation.

In [9]:
```
# figure, silhouette against k for the four linkages next to the share of points in the largest cluster
fig, ax = plt.subplots(1, 2, figsize = (11, 4.2))
for meth, c in zip(['ward', 'complete', 'average', 'single'], ['steelblue', 'darkorange', 'seagreen', 'indianred']):
    t = hac_tab[(hac_tab.space == 'ecams scaled') & (hac_tab.linkage == meth)]
    ax[0].plot(t['k'], t['silhouette'], 'o-', color = c, label = meth)
    ax[1].plot(t['k'], t['largest cluster share'], 'o-', color = c, label = meth)
ax[0].set_xlabel('clusters')
ax[0].set_ylabel('silhouette')
ax[0].set_title('ecams, silhouette by linkage')
ax[0].legend(fontsize = 8)
ax[1].set_xlabel('clusters')
ax[1].set_ylabel('share of points in largest cluster')
ax[1].set_title('ecams, how lopsided the cut is')
plt.tight_layout()
plt.savefig('figs/w11_linkage.png', dpi = 200)
plt.show()
```



The left panel shows the silhouette trap again. Average and single linkage have the highest silhouette on ECAMS (0.48 at three clusters) and the right panel shows their largest cluster holds 99 percent of the patients. Ward is the lowest in silhouette but it's the

only one where the largest cluster drops to 43 percent.

To sum up, density based clustering found no stable groups on either dataset, though its noise points are mildly enriched for delirium on ECAMS, and hierarchical clustering only gives useful groups with ward linkage, where the ECAMS clusters range from 45 to 98 percent delirious. Silhouette and cophenetic correlation both favored the linkages that gave one giant cluster, so I wouldn't pick a clustering from those numbers alone.